SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Degree

Major:

Networking, Cloud & IA (NCA)

Final Graduation Project Report

Theme:

Design and implementation of a network management platform for wireless mesh network routers

Realized by:

Oussama Taieb

Supervisors:

Mr. Mohamed-Bécha Kaâniche (SUP'COM)

Mr. Ahmed Ghamlouli (CODINTEK)

Work proposed and established in collaboration with:

CODINTEK

codintek

Future of wireless

Academic year : 2020-2021

Ecole Supérieure des Communications de Tunis

SUP'COM

2083 Cité Technologique des Communications - Elghazala - Ariana - Tunisie

Tél. +216 70 240 900 – site web : www.supcom.mincom.tn

DEDICATION

Dedicated to:

Everyone who has helped me be the person who I am today.

With love and respect.

ACKNOWLEDGMENTS

I'd like to express my gratitude to my supervisor Mr. Ahmed Ghemlouli a communication systems engineer and Mr. Fethi Fellah a telecommunication engineer for the guidance, encouragement and advice they provided me throughout my internship. This project could not have been completed without their enthusiastic participation and consistent survey. Furthermore, I am grateful to Mr. Mohamed-Bécha Kaâniche for his advice and support. It was an honor to be his student and a pleasure to know him.

ABSTRACT

Wireless networks are becoming increasingly popular as a result of radio communications technologies that have enabled them to replace traditional wired networks in applications ranging from video surveillance and smart cities to defense and emergency radio systems.
Typically, monitoring and managing such solution is of importance, as it is the case with the hosting company's wireless router product called Xtremesh. The scope of this project is to design and implement a network monitoring platform for these Xtremesh routers to help monitor and secure control of their systems.

Key Words:
Network Administration, Linux, Zabbix, Automation, Web Development, Grafana, Shell Scripting, SNMP, MIB, Nginx, REST API, JavaScript, MySQL, Virtualisation.

LIST OF ABBREVIATIONS

API: Application Programming Interface

CORS: Cross-Origin Resource Sharing

IPMI: Intelligent Platform Management Interface

JDBC: Java™ database connectivity

JMX: Java Management Extensions

LDAP: Lightweight Directory Access Protocol

LLD: Low Level Discovery

MIB: Management Information Base

NMS: Network Management System

OID: Object Identifier

OS: Operating System

SFTP: Secure File Transfer Protocol

SIEM: Security Information and Event Management

SMTP: Simple Mail Transfer Protocol

SNMP: Simple Network Management Protocol

SNR: Signal Noise Ratio

SSH: Secure Shell

WMI: Windows Management Instrumentation

XML: Extensible Markup Language

CONTENTS

Dedication i
Acknowledgments ii
Abstract iii
List of Abbreviations iv
List of Figures ix
List of Tables xii
General Introduction xiii
Chapter 1: Scope Statement 1
1.1 Introduction . . . 1
1.2 Project Charter . . . 1
1.2.1 Scope . . . 1
1.2.2 Constraints . . . 1
1.2.3 Objectives of the project . . . 2
1.2.4 Direction and benefits . . . 2
1.3 Project Owner . . . 2
1.3.1 Overview . . . 2
1.3.2 Expertise field . . . 3
1.3.3 Data sheet . . . 3
1.4 Conclusion . . . 3
Chapter 2: State of the art 4
2.1 Introduction . . . 4
2.2 Network Monitoring . . . 4
2.2.1 Network Monitoring Overview . . . 4
2.2.2 Network Monitoring Systems . . . 4
2.2.3 Monitoring Fields . . . 4
2.2.4 Exploitation in IT departments . . . 5
2.2.5 Open source vs Proprietary . . . 5
2.2.6 Agent-based vs Agent-less . . . 6
2.2.7 Auto discovery . . . 7
2.2.8 Low-level discovery . . . 7
2.2.9 Logical grouping . . . 8
2.3 Comparative description of open-source NMS . . . 9

2.3.1 Zabbix 9
2.3.2 OpenNMS 11
2.3.3 Icinga 13
2.4 NMS major drawback 14
2.5 Why Zabbix as the main NMS choice ? 15
2.6 Conclusion 15
Chapter 3: System Design **16**
3.1 Introduction 16
3.2 Architectural design 16
3.2.1 Deployment specifications 16
3.2.2 Xtremesh system 17
3.2.3 Zabbix architecture 17
3.2.4 Grafana 20
3.2.5 MySQL 22
3.2.6 Nginx 24
3.2.7 Simple Network Management Protocol SNMP 25
3.2.8 APIs 28
3.2.9 ReactJS 29
3.3 Conclusion 29
Chapter 4: Experimental Protocols **30**
4.1 Introduction 30
4.2 Working environment 30
4.2.1 Settings 30
4.2.2 Machine's initial configuration 30
4.3 Nginx installation 32
4.4 MySQL installation 33
4.5 Zabbix Installation 33
4.6 Grafana installation 37
4.7 Automation of Zabbix monitoring 40
4.7.1 Creation of Xtremesh's Zabbix template 41
4.7.2 Importing the template to Zabbix 47
4.7.3 Zabbix LLD rule (Low Level Discovery) 49

4.8 Xtremesh's dashboard model with Grafana 56
4.8.1 Steps 57
4.8.2 Result 61
4.9 Conclusion 62
Chapter 5: Xtremesh platform development **63**
5.1 Introduction 63
5.1.1 xMeshControl overview 63
5.2 Client-server model 63
5.3 Quality requirements 64
5.4 Platform architecture design 64
5.4.1 System design 65
5.4.2 Modular design 65
5.4.3 Authentication 67
5.4.4 Grafana charts automation 69
5.4.5 Project Assessment 73
5.5 Conclusion 77
General Conclusion **78**
References **R1**

LIST OF FIGURES

2.1 Zabbix logo 9
2.2 OpenNMS logo 11
2.3 Icinga logo 13

3.1 Deployment diagram 16
3.2 Zabbix components architecture 17
3.3 Remote vs local monitoring with Zabbix 18
3.4 Grafana Logo 20
3.5 Grafana Chart 20
3.6 Grafana dashboard example 21
3.7 Grafana Plugins 22
3.8 MySQL Logo 23
3.9 MySQL architecture 24
3.10 Nginx logo 24
3.11 Principle of SNMP communication 26
3.12 OID library opended in the MIB importer software 28
3.13 React Logo 29

4.1 CentOS 8 VM settings 30
4.2 Server IP address configuration 31
4.3 SELINUX configuration 32
4.4 Nginx service active 32
4.5 Zabbix server password configuration 34
4.6 Zabbix reverse proxy configuration 35
4.7 Zabbix-PHP timezone configuration 36
4.8 Zabbix login screen 36
4.9 Zabbix automation flow 41
4.10 Xtremesh MIB tree 43
4.11 MIB's structured data sample 44
4.12 Zabbix's templates list 48
4.13 Import Template to Zabbix 48

4.14 codintek-mib successful import . . . 49
4.15 Discovery check configuration . . . 50
4.16 SNMP OID check . . . 50
4.17 Discovery's configured SNMP checks . . . 51
4.18 Discovery Actions drop-down box . . . 51
4.19 Conditions to filter only the xmesh discovery checks . . . 52
4.20 Condition to filter only the Xtremesh devices . . . 53
4.21 Action form . . . 53
4.22 Add to xmeshGroup operation . . . 54
4.23 Link to template operation . . . 54
4.24 Operation field . . . 55
4.25 Monitored Xtremesh hosts . . . 55
4.26 Hosts latest data collected . . . 56
4.27 Zabbix data source configuration . . . 57
4.28 Graph panel configuration . . . 58
4.29 Graph options and styling . . . 59
4.30 Down-Stream RF graph for the node 192.168.1.137 . . . 60
4.31 192.168.1.137's Dashboard . . . 60
4.32 192.168.1.137's Dashboard Json model . . . 61

5.1 Client-server model . . . 64
5.2 System design . . . 65
5.3 Use case diagram . . . 66
5.4 Authentication process . . . 68
5.5 Request body function with host uid argument . . . 69
5.6 Dahboard creation API handler function . . . 70
5.7 Function that returns a parmetered JSON dashboard model . . . 71
5.8 Dashboard updating API handler function . . . 72
5.9 Dashboard creation success response . . . 72
5.10 Dashboard page . . . 73
5.11 Zabbix report page . . . 74
5.12 Zabbix report page . . . 74
5.13 Users page . . . 75

5.14 General Information page 75
5.15 Real time statistics page 76
5.16 RF configurations page 76

LIST OF TABLES

1.1 CODINTEK's sheet 3

GENERAL INTRODUCTION

Networks nowadays consist of hundreds to thousands of different devices. This fact explains why network monitoring systems are required to facilitate the task of networks administrators.

Even the most modern networks, however, cannot guarantee one hundred percent availability. From time to time, problems and failures will arise. Network monitoring is commonly utilized in order to be able to respond quickly to them.

In this context, I proposed a solution for monitoring and managing the hosting company's wireless routers. These routers called Xtremesh communicate with each other in a mesh network architecture. The aim of this project is to integrate and automate an open-source Network Management System (NMS), to develop a platform that is capable to configure, monitor and manage complex mesh networks.

This report is structured around five chapters. First I introduce the hosting company, its activities and the overall scope of the project. Second, I present the state of the art and realize a comparative study of the most used network management systems (NMS) then choose the best NMS tool to work with. Third, I explain the system design of this project's solution. Forth, I detail the experimental protocols, listing all important software and hardware materials needed to carry out the project. The final chapter demonstrates the implementation of Xtremesh platform and illustrates the process with screenshots.
Finally I conclude this report by over-viewing my work while presenting the pros and cons of the developed platform and exposing perspectives for future improvements.

CHAPTER 1
SCOPE STATEMENT

1.1 Introduction

In this chapter, I describe the major objectives and detail the project specifics, as well as the presentation of the hosting organization.

1.2 Project Charter

1.2.1 Scope

CodinTek, the project owner, is working to improve its private wireless network solution Xtremesh, which is designed to replace traditional wireless technologies such as WiFi and 4G/5G mobile networks when they are unable to meet the needs of customers.

Xtremesh routers have Beam Forming capabilities, network node mobility, a 20km operational range, and a throughput of 200MB/s.

The system can be used as a wireless infrastructure for video surveillance systems, large smart parking garages, and an emergency radio system. It is frequently deployed over a large area, replacing the traditional solution in which acquired data is transmitted via cables, and the large amount of manpower for deployment and maintenance is needed.

As it is the case for hundreds of nodes to be deployed, the systems of Xtremesh should be remotely maintained, updated, and monitored for a consistent functioning and productivity.

1.2.2 Constraints

Today the Xtremesh technology includes a demo network management tool in its application layer, realized by CodinTek team, only for a simple demonstrative monitoring and system debugging use.

This definitely poses serious problems when deploying the Xtremesh routers in terrains for serious production use, as the client needs to continuously keeps track and have full control over his infrastructure.

This situation implies the need for having to migrate to a powerful network management system that's capable of handling the deployment of the Xtremesh technology.

1.2.3 Objectives of the project

This project aims to integrate an open source NMS to develop a platform capable of providing an optimal monitoring and management solution for the Xtremesh network.

The network administrator should be given the possibility of not only the monitoring of a single node at a time, but also mapping the global status of all network nodes. He should be able to view system performance curves, collect run-time nodes information, change their configurations, keep track of historical data and manage the platform users. This solution should have no limitations neither in capabilities nor in amount of monitored devices.

1.2.4 Direction and benefits

As a result, this monitoring solution aids in the optimization of network performance and the reduction of network failures, as well as easily updating and adding features. The main benefits of an optimized network monitoring solution for the Xtremesh system are lesser infrastructure problems, increased productivity, and fast and reliable data flow.

1.3 Project Owner

1.3.1 Overview

CodinTek, founded in 2007, is a global hardware and software services and solutions provider. It is a global provider and developer of cutting-edge wireless broadband solutions and sensor networks, and innovative product design services as well as turnkey radio frequency broadband network solutions.

In addition, CodinTek engineers are specialized and experienced at designing wireless solutions for IOT. They provide all-in-one modules, tools and kits for smart-cities and industry. [1]

1.3.2 Expertise field

CodinTek develops communication systems, its engineering team has considerable expertise and experience in the design and implementation of complex wireless communication systems aspects such as high performance, security, long range and low power consumption.

Giving technical expertise and reference products, CodinTek continues to be successfully retained by companies from different countries mainly the USA, Korea, France, Kingdom of Saudi Arabia.

1.3.3 Data sheet

Company name	CODINTEK
Business sector	Various Industries, research and defense applications.
Legal form	SARL
CEO	Mr. TLICH Mohamed
Date of creation	2007
Email	info@codintek.com
Phone number	+21697444843
Web site	http://www.codintek.com
Location	Codintek – Elgazela Technopark, Building I1, 4th Floor – 2088 Ariana– Tunisia

TABLE 1.1 CODINTEK's sheet

1.4 Conclusion

In this chapter I briefly presented the reasons for undertaking the project, its goals and a shared understanding of its direction. I also introduced CodinTek, the hosting company and the owner of the project. The next chapter will be dedicated to the state of the art study of my solution.

CHAPTER 2
STATE OF THE ART

2.1 Introduction

During this chapter, I provide a state of the art study of network monitoring and its exploitation in IT fields. A comparative analysis between the existing open-source NMS tools will take place in order to choose the best tool to work with.

2.2 Network Monitoring

2.2.1 Network Monitoring Overview

Network monitoring is a mechanism that allows network administrators to track the status of a computer network in real time. Network monitoring is a critical IT process that ensures the proper operation of network components such as firewalls, servers, routers, and switches by continuously evaluating their performance in order to maintain the availability of their services.[2]

2.2.2 Network Monitoring Systems

A network monitoring system (NMS) detects slow network performance or non-functioning network devices. The monitoring is based on an examination of throughput, error rates, packet loss, latency, network node availability, and response time. If a failure occurs, the network administrator is notified of the failure via a warning banner, email, phone, and other alarms. Servers, routers, switches, and even end devices are all supported by network monitoring. Furthermore, it could be used in any type of network, including WLAN, LAN, VPN, and even WAN. [3]

2.2.3 Monitoring Fields

Network monitoring tools perform a variety of scanning and analyzing tasks for various types of devices and services. This was accomplished through the use of various protocols operating on different OSI layers. Its fields can be classified by monitoring topic such as networks, clouds, servers, web, etc...

2.2.4 Exploitation in IT departments

The use of network monitoring tools frees network administrators from routine tasks and allows them to focus on more important tasks. They will have a complete picture of network processes at any given time. Furthermore, routine failure troubleshooting could be atomized by a special script.

Without a doubt, some unexpected failures must still be handled manually. However, because of the detailed diagnosis of the problem, it will be much easier.

When it comes to selecting a network monitoring tool, network administrators face a significant challenge. There are currently more than 50 tools available on the market from various vendors.

2.2.5 Open source vs Proprietary

There are two types of monitoring applications available on the market today. The first is a monitoring tool built with open source code. Typically, such systems are distributed under the GPLv2 license. As a result, third-party developers are permitted to make code-level changes. The second type of monitoring tool is proprietary monitoring software. The license prohibits any kind of code-level modification.

In my opinion, open source network monitoring provides significantly more benefits than proprietary network monitoring. The amount of capabilities brought by open systems will benefit the end user.

Despite the fact that network monitoring vendors strive to include the most relevant monitoring features, it is nearly impossible to create a single solution that is optimal for any network. Different networks have distinct requirements.

This is where the open source network monitoring tool comes into play. If a feature relevant to network monitoring is not included in the version out of the box, it can be created by the network administrator or downloaded from the community with sufficient skills and knowledge.

Overall, open source network monitoring tools provide significant benefits for network monitoring and help to expand some features that vendors do not provide.

2.2.6 Agent-based vs Agent-less

Another decision for system administrators is whether to use agent-based or agentless network monitoring. There is no single best solution for every network. It is determined by the expected level of monitoring, the type of network, and even the budget.

Agent-based monitoring is comprised of a piece of software known as the agent. The agent is a program that runs locally on servers and other network devices. Its purpose is to keep track of network performance. A warning message is generated if a failure occurs. Furthermore, the agent could troubleshoot some errors.
Agent is a small application. However, some of them may consume a significant amount of network resources. As a result, the concept of using network monitoring is becoming obsolete. That is why the lightweight agent, also known as the "invisible" agent, is gaining popularity these days.
The primary advantage of agent-based monitoring is that it provides more in-depth network analytics. Furthermore, agent-based monitoring tools may be able to diagnose hardware performance. It also has alerting and reporting capabilities. Some errors may be automatically troubleshooted.
The main disadvantage is that deploying such a system is a time-consuming process in which many network details must be considered. Furthermore, the agents should be updated. Traditional agent-based solutions may have an impact on network performance. It should also be noted that agent-based monitoring tools have a significantly higher license cost.

Agent-less is a solution that does not require the installation of any additional agents. Direct packet monitoring is used in network analysis. It was used to keep track of network availability and performance. It does not, however, provide any specific information about the failure.
Agent-less monitoring is typically based on SNMP (Simple Network Monitoring Protocol) or WMI (Windows Management Instrumentation) (Windows Management Instrumentation). It is built around a central management station that keeps track of all

network devices. Agent-based solutions, on the other hand, provide more detailed metrics.
The main advantage of using agent-less monitoring is that no agent is required. As a result, there is no impact on network performance. The deployment procedure has been simplified. Furthermore, there will be no need for the agent to be updated on a regular basis. Furthermore, the cost is relatively lower. The most significant disadvantage of using agent-less monitoring is the lack of detailed metrics.

Both agent-based and agent-less network monitoring are recommended. In today's market, vendors such as Nagios and Zabbix offer both agent-based and agent-less capabilities in their solutions. Agent-based monitoring could be used in critical parts of the network where availability and performance are prioritized. For less critical parts of the network, agent-less is preferable.[4]

2.2.7 Auto discovery

One of the challenges for a network manager is keeping the network management system up to date in the face of constant change. In most network environments, new devices are added on a weekly, if not daily, basis. Auto discovery is used to keep track of the constantly changing environments.

Auto discovery is a feature that allows you to perform a network element search. Furthermore, it automatically adds new devices and removes those that are no longer connected to the network. It also discovers network interfaces, ports, and file systems.

Despite the fact that auto discovery is critical in network monitoring, some tools on the market do not include it. As a result, when selecting a network monitoring tool, network administrators should look for the presence of auto discovery.[5]

2.2.8 Low-level discovery

Low-Level Discovery (LLD) is used to monitor file systems and network interfaces without the need to manually create and add each element. Low-level discovery is a dynamic feature that adds and removes elements automatically. It also generates triggers and graphs for file systems, network interfaces, and SNMP tables automatically.

Templates were in use prior to the widespread acceptance of LLD. Creating a template, on the other hand, is a time-consuming process. Each template should create a trigger by hand for each port or logical disk. Furthermore, it is necessary to specify what the trigger should be aware of.

For example, A switch with 24 ports is used in a network. The template should be created using SNMPv2 and IF-MIB triggers to monitor port status. Of course, depending on network policies, up to 14 elements could be created. Following that, a template should be duplicated 23 times, once for each port. It should also be noted that networks are made up of numerous switches and other devices.That is why it takes even longer to spread triggers across all network devices. For a network manager, this is a routine and time-consuming process.

Instead of having to manually create templates for each port, LLD allows you to create data element and trigger prototypes only once. Following that, LLD discovers ports, file systems, and SNMP tables automatically. Several different triggers could be developed to meet various needs.

Overall, low-level discovery aids in network monitoring optimization by providing alerting and graphical output of port status, logical disks, and SNMP tables.

2.2.9 Logical grouping

It is difficult to monitor and troubleshoot all of the devices in large networks with many devices during dynamic network monitoring. Logical grouping enables the combination of devices of the same type. As a result, logical grouping greatly simplifies network monitoring at the enterprise level.

Logical grouping enables the grouping of network devices of the same type. What should be monitored and what actions should be taken in the event of a failure could be defined for each group. Furthermore, by utilizing logical grouping, it is possible to configure unified settings for all members of the group. An alert is displayed if one or more members of the group are down or offline.

To summarize, logical grouping aids system administrators in monitoring the status of devices. Almost all current network-monitoring tools support logical grouping. [6]

2.3 Comparative description of open-source NMS

The section that follows compares general and technical information for a number of well-known open source network monitoring systems. I've pitted the best open source network monitoring softwares against each other in order to choose the best one to work with.

2.3.1 Zabbix

An open source software that monitors various network parameters and entities as well as the health and integrity of servers. It employs a versatile notification mechanism that enables users to set up e-mail-based alerts for virtually any event. Rapid response to network nodes issues. Based on the stored data, it provides excellent reporting and data handling features. This is ideal for capacity planning.



FIGURE 2.1 Zabbix logo

Architecture:

It is compatible with both agent-based and agent-less system architectures. In agent-based mode, the agent must be installed on the client host to be monitored by the Zabbix server. To collect metrics in agent-less mode, devices must support the SNMP protocol.

Features:

- Metrics can be collected from any device, system or application, using a multi-platform Zabbix agent, SNMP and IPMI agents, or a combination of the two. Agent-less user service monitoring, custom methods, calculation and aggregation, end-user web monitoring.
- Automatically detect problem states in the incoming metric flow. There's no need to keep an eye on incoming metrics all the time. Definition options that are ex-

tremely versatile, Separate the problem conditions from the resolution conditions. There are several levels of severity. Anomaly detection, root cause analysis Trend forecasting

- Effortless deployment, by use out-of-the-box templates to save time. Zabbix can be installed in a matter of minutes. For the majority of popular platforms, use out-of-the-box templates. Create your own templates. Utilize hundreds of templates created by the Zabbix community. Request a template building service from the Zabbix team. Using configuration templates, you can monitor thousands of similar devices.
- Auto discovery feature makes it easy to monitor large, dynamic environments automatically. Take automatic actions when elements are added, removed, or changed. Periodically scans network and discovers device type, IP, status, up-time/down-time, etc..., and takes predefined actions.
- Low level discovery (LLD) to creates items, triggers, and graphs for various elements on a device automatically.
- Scale without bounds, as it creates a distributed monitoring solution while maintaining centralized control. Data should be collected from thousands of monitored devices.
- Run custom scripts on monitored hosts from a remote location.[7]

Security:

On all levels, it protects your data. All Zabbix components are securely encrypted. Various methods of authentication: Open LDAP and Active Directory. Schema of user permissions that is adaptable Zabbix code is available for security audits.

Supported OS:

Zabbix server can only be installed on Linux servers. Zabbix agents are supported in Linux, Solaris, HP-UX, AIX, Free BSD, Open BSD, OS X, Tru64/OSF1, Windows NT4.0, Windows 2000, Windows 2003, Windows XP, Windows Vista.

Technologies used:

Zabbix is built around modern Nginx or Apache web server, leading database engines such as MySQL and PostgreSQL, C and PHP scripting languages.

(PHP 45.3%, C 43.2%, Go 4.0%, JavaScript 2.8%, CSS 2.5%, SCSS 0.7%, Other 1.5%).

API usability:

Zabbix can be integrated into any aspect of the IT environment. Through the Zabbix API, you can gain access to all Zabbix functionality from external applications. Zabbix management can be automated using one of the 200+ API methods available, as well as the creation of new Zabbix-compatible applications. Zabbix can be easily be integrated with third-party software and historical data can be retrieved and managed.

2.3.2 OpenNMS

OpenNMS is an enterprise-grade, open-source platform for developing network monitoring solutions. Among the goals are to reduce time to production by supporting industry standard network management protocols, agents, and a programmable provisioning system.

The OpenNMS contributes to the development of interoperable network monitoring solutions. Meridian and Horizon are the two versions available. Horizon is released more frequently and includes the most recent available features, whereas Meridian is a longer-term release (about every 12 months) that does not include the most recent features until they have proven stable in previous Horizon releases.[8]



FIGURE 2.2 OpenNMS logo

Architecture:

An event-driven architecture enables the flexible integration of workflows into ex-

isting monitoring and management stacks. OpenNMS standardizes device and vendor specific messages, as well as protocol-specific performance metrics. The data, which is based on open source technologies, is accessible via a powerful REST API and can be used in high-level management workflow applications.

Features:

- Extensive discovery of networks, applications, services, and topology
- Performance and fault management.
- Network traffic analysis, route monitoring
- Measure performance using open protocols and management agents (SNMP, JMX, WMI, WS-Management, NSClient++, HTTP, XML, JSON, JDBC). Using correlation mechanisms, receive SNMP Traps and Syslog messages and convert them to high-level alarms.
- Support for email and SMS notifications.
- Event-Driven Architecture that is Highly Scalable.

Security:

The ability to assign user and group permissions, as well as security roles, is enabled. The Data Options module gathers anonymous statistics and publishes them at system startup and every 24 hours thereafter. Data encryption and authentication measures are in place.

Supported OS:

OpenNMS, like most open-source software, was designed for your preferred flavor of Linux, but Windows and OSX are also supported.

Technologies used:

A Java bases application that stores data in the PostgreSQL database.
(Java 84.7%, HTML 3.3%, JavaScript 10.6%, CSS 0.3%, Shell 0.5%, Perl0.2%, Other 0.4%)

API usability:

A wide range of REST APIs for metrics collection, visualization, graphs APIs, and

so on. Allows you to connect to the OpenNMS network monitoring platform. In the future, support for other common REST operations such as creating, updating, deleting, and importing requisitions is planned.

2.3.3 Icinga

Icinga is an open source network monitoring system that checks network resource availability, notifies users of outages, and generates performance data for reporting. Icinga, which is scalable and extensible, can monitor large, complex environments in multiple locations.



FIGURE 2.3 Icinga logo

Architecture:

Icinga 2 is the monitoring server, and it requires Icinga Web 2 to be installed on top of your Icinga Stack. The configuration can be easily managed within the Icinga DSL using the Icinga Director, configuration management tools, or plain text.[9]

Features:

- Availability, performance, and trends to always be aware of the current state of everything.
- Icinga can Manage a large number of monitoring objects. Automatic import and synchronization of all types of data is used use via the Director or other modules.
- Can be used to keep track of host and service availability Hosts and services can be continuously checked such as services for networks (HTTP, SMTP, SNMP, SSH, etc.)
- When it comes to incident resolution, having knowledge is the first step. Icinga notifies you when it makes sense via any channel you specify.
- Icinga collects and sends data from and to many of your existing DevOps tools, allowing you to build a monitoring solution that meets your exact requirements.

Security:

Access control is a critical component of securely configuring Icinga Web 2. It is critical to note that not every user with access to Icinga Web 2 can perform any action or see any host or service. To avoid mis-configuration and security breaches, only allow a small group of administrators to change the Icinga Web 2 configuration.
Icinga 2 comes with its own SELinux policy. icinga2-selinux is a Red Hat Enterprise Linux 7 and derivatives policy package. The package executes the targeted policy, which restricts Icinga 2's enabled features and running commands.

Supported OS:

Cross-platform (master on Linux only)

Technologies used:

Basically written in C++ and PHP. (C++ 91.0%, CMake 2.7%, PL/pgSQL 2.3%, Yacc 1.4%,C# 0.7%, Shell 0.6%, Other 1.3%)

API usability:

Using HTTP requests, the Icinga 2 API allows you to manage configuration objects and resources in a simple, programmatic manner. The URL endpoints are logically separated, making it simple to query, create, modify, and delete configuration objects.

2.4 NMS major drawback

A new comer to the network management systems tools might get overwhelmed by the its complicated implementation, configuration and tuning. Absolutely nothing works unless specifically configured to do so. Basic stuff such as Auto Discovery, Low Level Discovery, Logical Grouping, etc... isn't automated and requires some knowledge to fire up quickly the NMS processes.

That's why this project is the solution for a fast deployed, easy-to-use and pre-configured network monitoring platform that will handle all processes automation and the open-source NMS powerful features, via direct API connection. This will make the monitoring experience much easier, analytically more efficient and sets up to interpret the collected metrics through well designed charts and graphs.

2.5 Why Zabbix as the main NMS choice ?

Zabbix meets 90 percent of the requirements for a reliable network monitoring tool. It monitors both agent-based and agent-less systems. Low-level Discovery, Auto-Discovery, and Logical grouping are examples of features that could be found.

One of Zabbix's main advantages is its scalability, which allows it to be used in environments of any size. The scalability principle is applied to performance and usability. In addition, Zabbix is a trustworthy and predictable network monitoring software. If Zabbix alerts the user to some failures, he or she can be certain that such a problem exists.

Also, one of the important aspects i have studied is the power of Zabbix's Api. through it, i can get access to all Zabbix functionalities from any external application, with more than 200 methods available for use, with a clean structured documentation.

Community wise, Zabbix developers frequent the Forums, where they assist Users in resolving the most difficult issues and answering the most difficult questions. Social media groups and blogs are always available to answer and post about anything and everything zabbix-related. Thousands of companies worldwide, including DELL, Salesforce, ICANN, Orange, and others, use it.

2.6 Conclusion

During this chapter, I explained the overall context of network monitoring and did a comparison between some best open source NMS tools provided in the market. After that comparative analysis, I chose to work with Zabbix which I believe it's best suited for this project's needs.

CHAPTER 3
SYSTEM DESIGN

3.1 Introduction

In this chapter, I emphasize the design of the system architecture and share a basic understanding of component parts and their subsequent interaction with one another.

3.2 Architectural design

3.2.1 Deployment specifications

The diagram below shows what hardware and software components exist, and how the different pieces are connected.

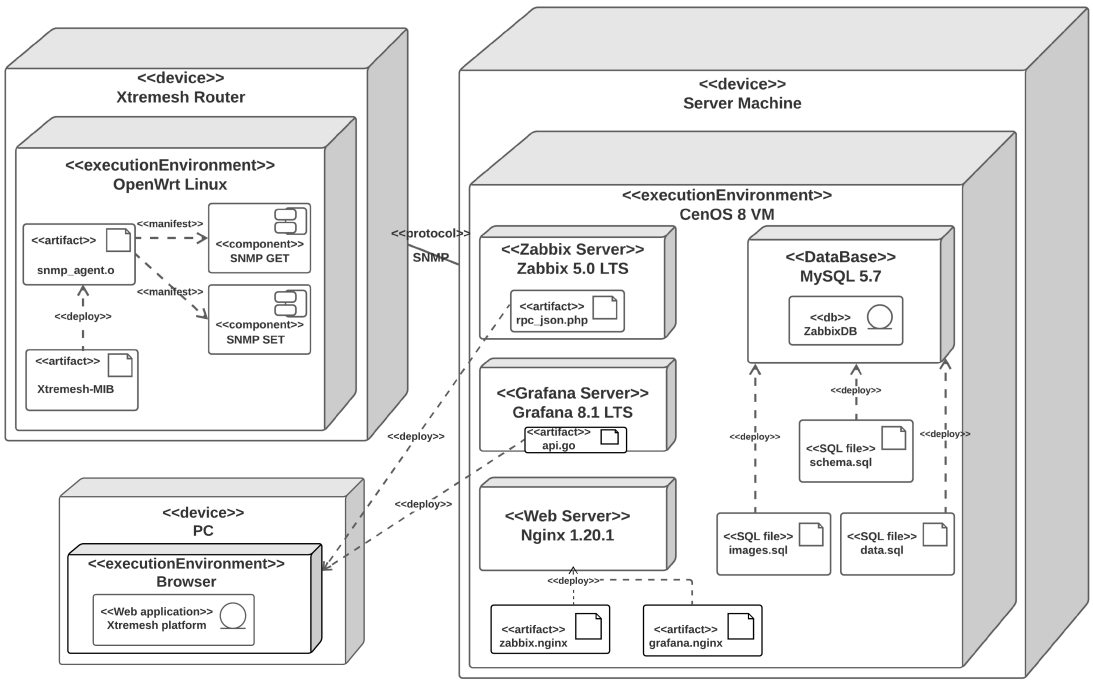


FIGURE 3.1 Deployment diagram

The system will operate in a virtual environment, a dedicated CentOS 8 server machine running on top of a physical machine that belongs in the same local network

where Xtremesh nodes are deployed. Zabbix will collect data from the network's Xtremesh devices using the SNMP protocol and store it in the MySQL database. Grafana dashboards will be created and embedded in the web application using JavaScript code that handles Grafana API. Nginx, a dedicated web server, will handle the load balancing and API connections parameters for optimizing performance and security settings such as CORS policies. Nginx will be required to handle communication between the Xtremesh platform and the services running in the virtual machine.

3.2.2 Xtremesh system

The Xtremesh system has two main modules at the highest abstraction level, the Data Plane and the Control Plane. The Data Plane of the protocol stack is made up of entities that are directly involved in the transfer of user payload. The Control Plane is made up of entities that are involved in controlling and managing the system. The management module for the user Interface will be comprised of the protocols SNMP, SSH, and SFTP.

3.2.3 Zabbix architecture

Zabbix is made up of several major software components, the network diagram below illustrates how they interact with each other.

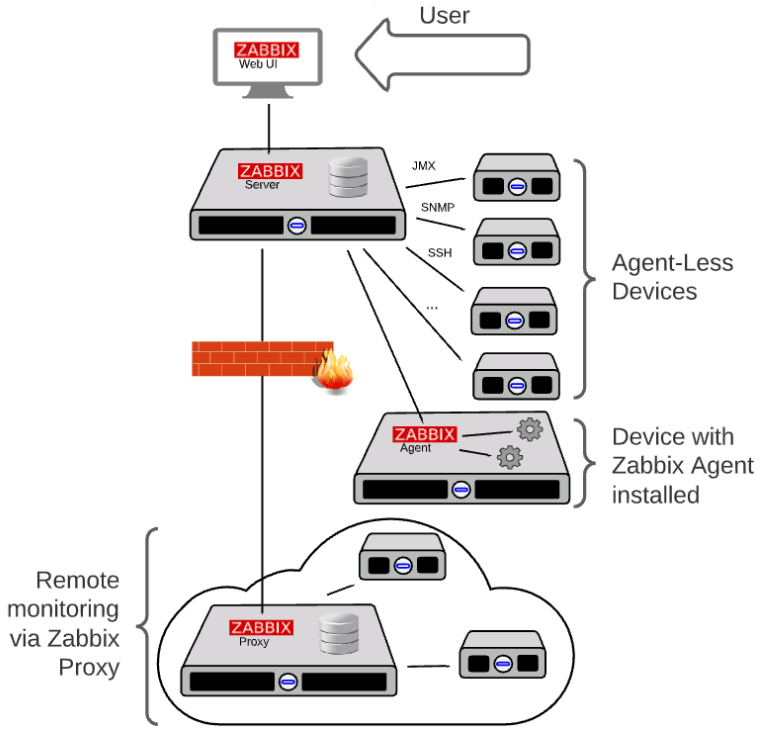


FIGURE 3.2 Zabbix components architecture

Zabbix server

This is the Zabbix software's core. The Server can use simple service checks to remotely check networked services and devices, but it is also the central component to which the Agents will report availability and integrity information and statistics.

The Server is the central repository for all configuration, statistical, and operational data, and it is the entity in the Zabbix software that actively alerts administrators when problems occur in any of the monitored systems. Zabbix can also monitor network devices using SNMP and perform agent-less monitoring.

Zabbix proxy

The Proxy is an optional component of the Zabbix installation. The Proxy collects performance and availability data on Zabbix Server's behalf. All collected data is buffered locally before being transferred to the Zabbix Server to which the Proxy belongs.

Zabbix Proxy is an excellent solution for centralized monitoring of remote locations, branches, and networks that do not have local administrators.

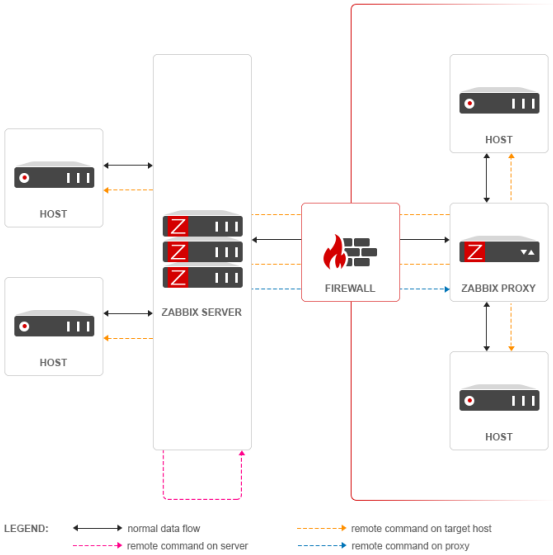


FIGURE 3.3 Remote vs local monitoring with Zabbix

Zabbix agent

The Zabbix Agent is a small program that will collect operational data from the system on which it is running and send it to Zabbix for further processing. In the event of a failure (such as a full hard disk or a crashed service process), the Zabbix Server can actively alert the administrators of the machine that reported the failure. Because they use native system calls to gather statistical data, Zabbix Agents are extremely efficient.

Zabbix template

A template is a grouping of entities that can be applied to multiple hosts with ease. Items, applications, triggers, low-level discovery rules, Web scenarios, dashboards and graphs could be the entities. Because many hosts in real life are identical or very similar, the set of entities (items, triggers, graphs, etc.) created for one host may be useful for many others. Of course, you could manually copy them to each new host, but that would be time-consuming. Instead, copy them to a template and then apply the template to as many hosts as necessary.

When a template is linked to a host, all of the template's entities (items, triggers, graphs, etc.) are added to the host. Templates are assigned to each individual host on an individual basis (and not to a host group).

Templates are frequently used to group entities for specific services or applications (such as Apache, MySQL, PostgreSQL, Postfix...) and are then applied to hosts that run those services.

Another advantage of using templates is when something needs to be changed for all hosts at the same time. Changing something at the template level once will cause the change to be propagated to all linked hosts. As a result, using templates is an excellent way to reduce one's workload and streamline Zabbix configuration.[10]

3.2.4 Grafana

Grafana is an open-source analytics and interactive visualization web application. Thousands of businesses rely on it to monitor metrics, power plants, logs, beehives, applications, and custom data sources. When connected to supported data sources (such as Zabbix data source in this use case), it generates charts, graphs, and alerts for the web that provide valuable insights. It can be expanded via a plug-in system, and end users can create advanced monitoring dashboards using the interactive query builders it provides.



FIGURE 3.4 Grafana Logo

Grafana is a popular visualization tool in monitoring stacks, frequently used in conjunction with time series databases such as InfluxDB, Prometheus, and Graphite. Sensu, Icinga, Zabbix, Netdata, and PRTG are examples of monitoring platforms. SIEMs like Elasticsearch and Splunk, as well as other data sources Grafana's user interface was initially based on Kibana version 3.[11]

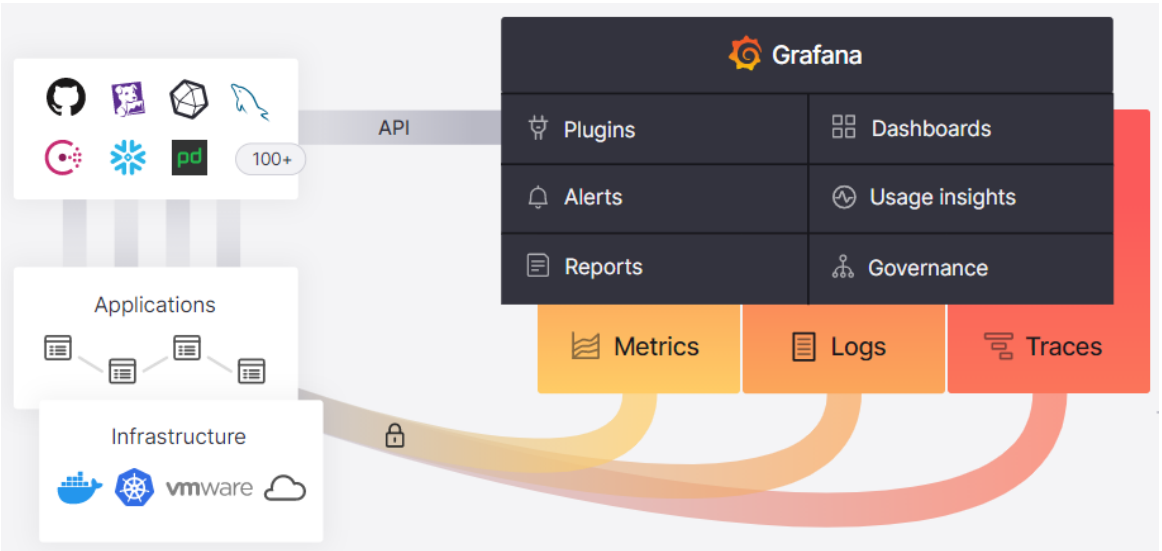


FIGURE 3.5 Grafana Chart

Grafana allows anyone to create and share dynamic dashboards that promote collaboration and transparency.

Dashboards

Grafana dashboards do not only provide insightful meaning to data collected from multiple sources, but team members can also share the dashboards created, allowing them to explore the data together.



FIGURE 3.6 Grafana dashboard example

Plugins

Grafana plugins allows to connect tools and applications. Data source plugins use APIs to connect to existing data sources and render the data in real time, eliminating the need to migrate or ingest the data.

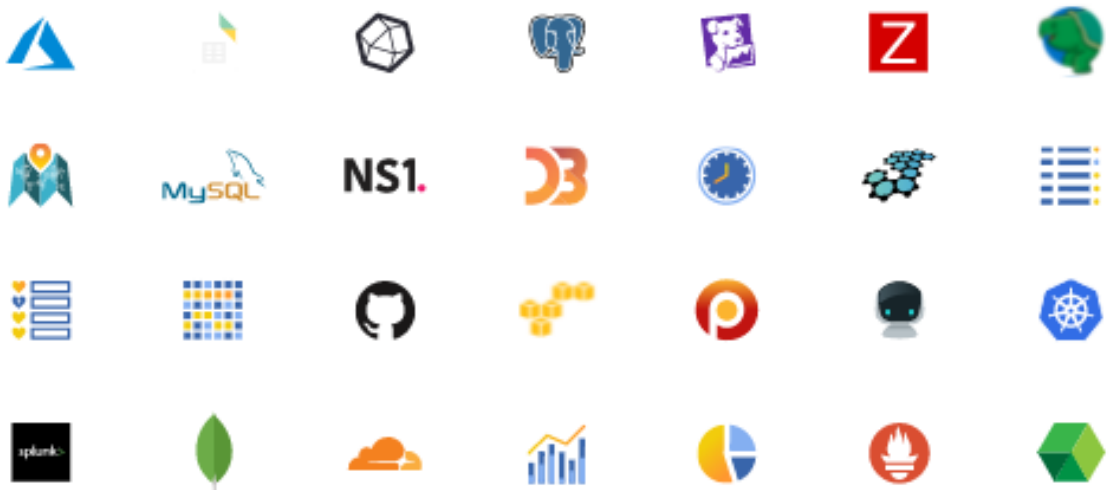


FIGURE 3.7 Grafana Plugins

- Data source plugins interact with external data sources and return data in a format that Grafana understands. data can immediately be used in any existing dashboards by adding a data source plugin. When importing data from other systems, usage of data source plugins is mandatory.
- App plugins, such as the Prometheus, Zabbix, Kubernetes and many other apps, combine data sources and panels to provide a unified experience. When wanting to create a unique, out-of-the-box monitoring experience, Grafana uses app plugins.
- Panel plugins enables adding new visualizations to dashboard, such as maps, clocks, pie charts, lists, and more. When wanting to visualize data returned by data source queries, navigate between dashboards, or control external systems, usage of panel plugins is very helpful.

3.2.5 MySQL

MySQL is a relational database management system that is available as free open source software under the GNU General Public License. Oracle Corporation also backs it up.

It is a database management system that is quick, scalable, and simple to use. It's is compatible with a wide range of operating systems, including Windows, Linux, and MacOS.



FIGURE 3.8 MySQL Logo

Mainly a Structured Query Language that is used to manipulate, manage, and retrieve data through the use of Queries. [12]

It's architecture describes the relationship between the various components of its system. MySQL adheres to the Client-Server Architecture. It is designed so that end users, or clients, can access resources from a computer, or server, by utilizing various networking services. The architecture contains the following major cores :

- **Client:**
  In the diagram below, this is the topmost layer. With the help of the Client Layer, the Client sends request instructions to the Serve. The client submits a request using valid MySQL commands and expressions via Command Prompt or GUI screen. If the expressions and commands are correct, the output appears on the screen.
- **Server:**
  The second layer of MySQL architecture is in charge of all logical functions of MySQL's relational database management system. This MySQL System Layer is also known as the "Brain of MySQL Architecture." When the client sends request instructions to the server, the server responds with the output as soon as the instructions are matched.
- **Storage layer:**
  This Storage Engine Layer of MySQL Architecture distinguishes it and makes it appealing to developers. Because of this layer, MySQL is regarded as the most widely used RDBMS. In MySQL server, different types of storage engines are used for different situations and requirements, such as InnoDB, MYiSAM, NDB, Memory, and so on. These storage engines are used as pluggable storage engines, into which user-created tables are plugged.

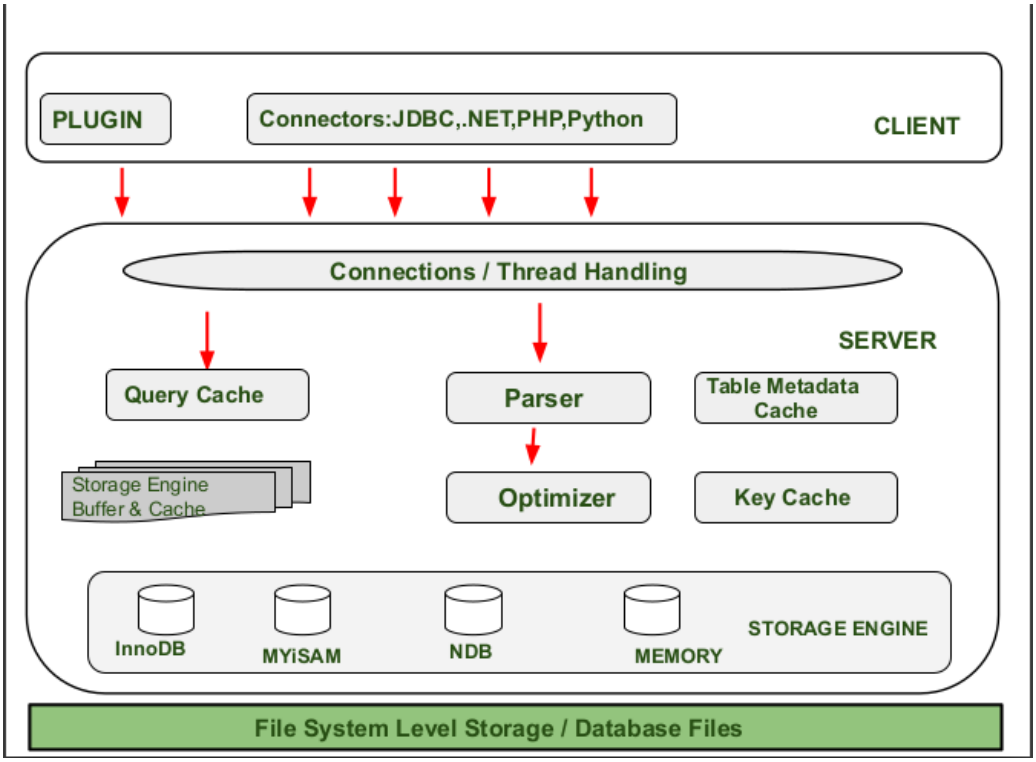


FIGURE 3.9 MySQL architecture

3.2.6 Nginx

NGINX is a high-performance web server and reverse proxy that is free and open source. Well known for its high performance, stability, extensive feature set, ease of use, and low resource consumption.

In a nutshell, it's a Linux based open source software that can be used for web serving, reverse proxying, caching, load balancing, media streaming, and other purposes.



FIGURE 3.10 Nginx logo

Nginx is a dedicated web server that has solved efficiency issues and provides the best ways to handle thousands of concurrent requests. In our use case, I will develop a platform that must be capable of not having a difficult time handling all of the requests

and not experiencing down times where concurrent requests are handled from multiple APIs connections at a time.[13]

3.2.7 Simple Network Management Protocol SNMP

The Simple Network Management Protocol (SNMP) is a networking protocol that is used in Internet Protocol networks to manage and monitor network-connected devices. The SNMP protocol is embedded in a variety of local devices, including routers, switches, servers, firewalls, and wireless access points that can be accessed via their IP address.

For network monitoring, SNMP is widely used in network management. SNMP exposes management data on managed systems in the form of variables called object identifiers (OID) organized in a management information base (MIB) that describe the system status and configuration. These variables can then be queried (and, in some cases, manipulated) remotely by management applications. [14]

SNMP communication

In typical SNMP usage, one or more administrative computers known as managers are tasked with monitoring or managing a group of hosts or devices on a computer network. Each managed system runs a software component known as an agent, which sends information to the manager via SNMP.
An SNMP-managed network is made up of three major components:

- Managed devices
- Agent - piece of software that run on managed devices.
- NMS (Network Management Station) – software that runs on the manager's computer.

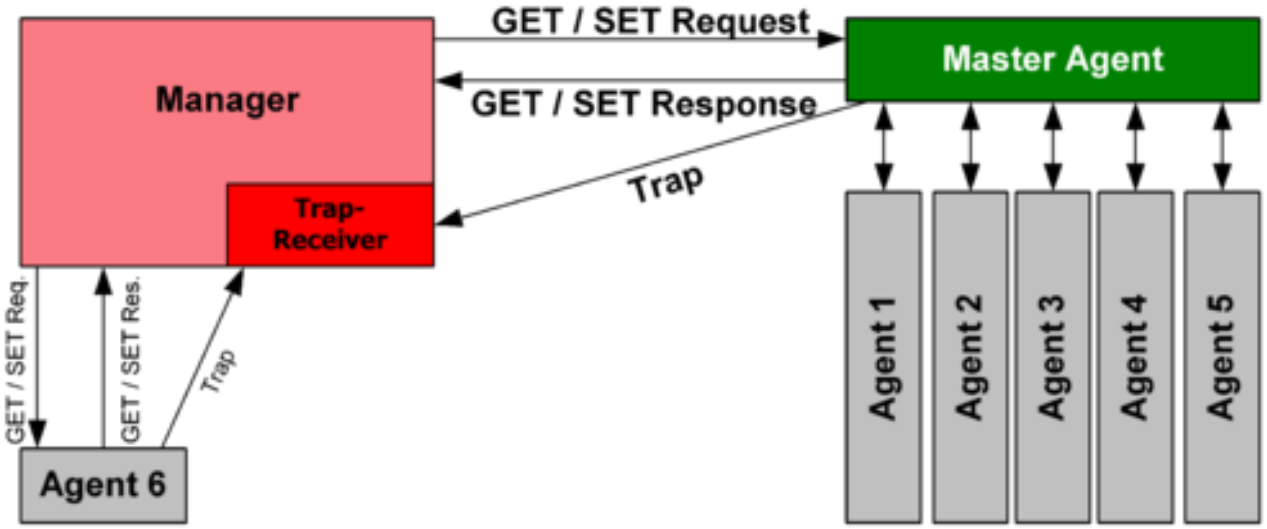


FIGURE 3.11 Principle of SNMP communication

A managed device is a network node that has an SNMP interface that allows unidirectional (read-only) or bidirectional (read and write) access to node-specific data. Managed devices communicate with NMSs about node-specific information. The managed devices, also known as network elements, can be any type of device, such as routers, access servers, switches, cable modems, bridges, hubs, IP telephones, IP video cameras, computer hosts, and printers.

A network-management software module that resides on a managed device is known as an agent. An agent understands management information on a local level and converts it to or from an SNMP-specific format.

Applications that monitor and control managed devices are run by a network management station. The majority of the processing and memory resources required for network management are provided by NMSs. Any managed network may have one or more NMSs.

Management Information Base (MIB)

MIB is an acronym for Management Information Base, and it refers to a collection of information that is organized hierarchically. These are accessed via a protocol like SNMP. MIBs are classified into two types: scalar and tabular. Tabular objects define multiple related object instances grouped in MIB tables, whereas scalar objects define a single object instance. MIBs are definition collections that define the properties of the managed object within the device being managed.

For example, on a switch, the typical objects of interest are the incoming and outgoing traffic, as well as the rate of package loss or the number of packets addressed to a broadcast address.

Object Identifiers (OIDs)

OIDs are abbreviated for Object Identifiers. OIDs are used to uniquely identify managed objects within a MIB hierarchy. This can be represented as a tree, with different organizations assigning different levels to it. Top level MIB object IDs (OIDs) are assigned to various standard organizations. Private branches, including managed objects, are defined by vendors for their own products.

They are typically provided by hardware manufacturers or can be found in so-called OID repositories, which contain collections of MIB trees and the corresponding OIDs. [15]

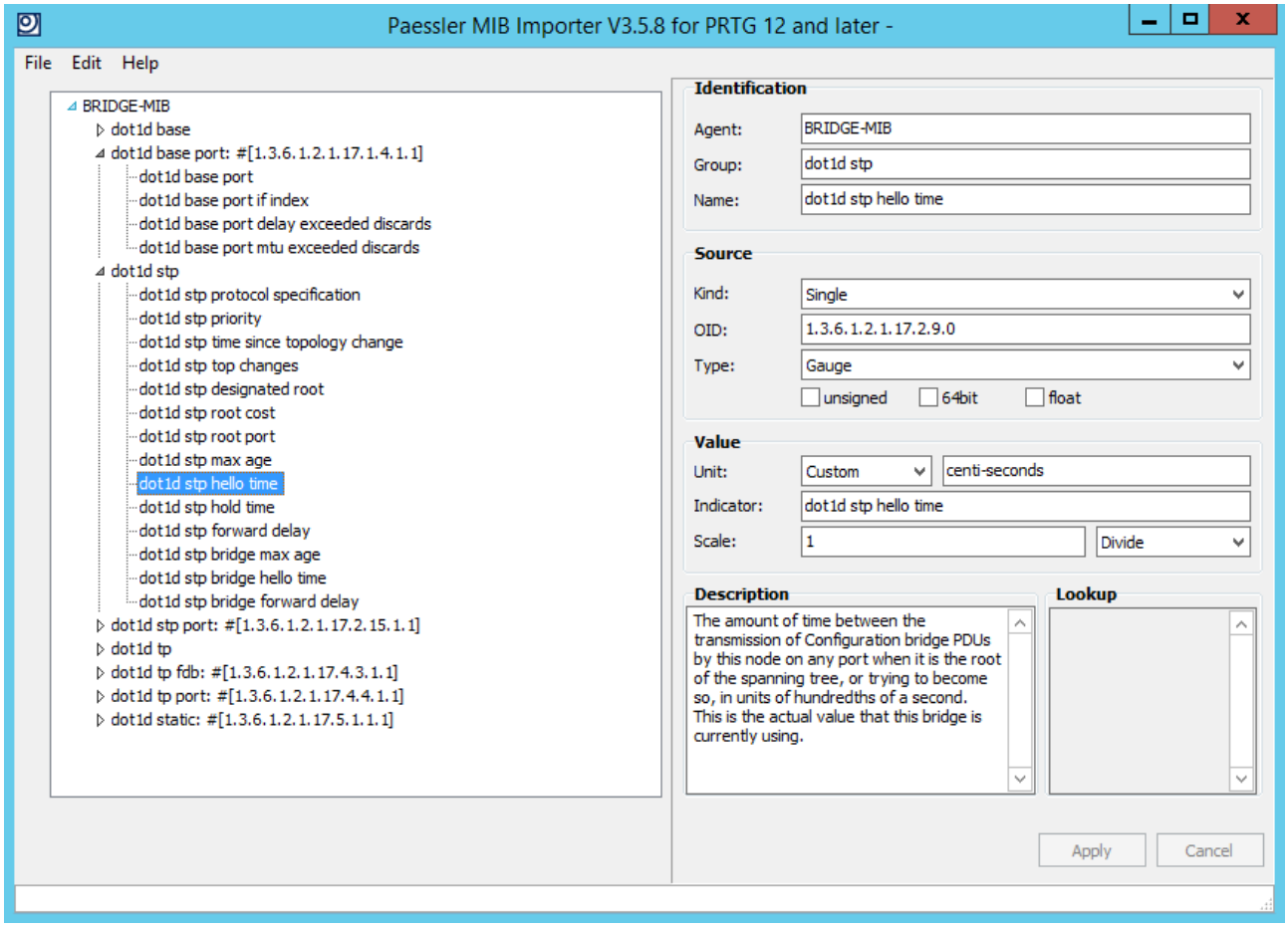


FIGURE 3.12 OID library opended in the MIB importer software

3.2.8 APIs

API stands for Application Programming Interface, and it is a software intermediary that allows two applications to communicate with one another. Zabbix and Grafana services are the third party applications to be configured, integrated and automated in the web interface code base. That's where the power of their APIs come in handy.

Zabbix API

The Zabbix API allows for programmatic retrieval and modification of Zabbix configuration, as well as access to historical data. It's widely used for creating new Zabbix applications, integrating Zabbix with third-party software, and automating routine tasks. It is a web-based API that is integrated into the web frontend.

It employs the JSON-RPC 2.0 protocol, which implies two things. The API consists of several distinct methods, and the requests-responses between clients and the API are encoded in JSON format. The API consists of a number of methods that are nominally separated into different APIs. Each method only accomplishes one task.

Grafana API

The Grafana backend exposes an HTTP API, which is the same API used by the frontend to save dashboards, create users, and update data sources.
The API can be accessed using one of two authentication methods: Through basic authentication, a Grafana Admin user can access some parts of the Grafana API. Or an API Token is used to access all organization actions. An API Token is associated with a company. It can be used to build dashboards and other components that are unique to that organization.

3.2.9 ReactJS

React is a JavaScript library used to create user interfaces. Makes it simple to create interactive user interfaces. Create simple views for each state of the application, and React will update and render only the necessary components when the data changes.



FIGURE 3.13 React Logo

React creates encapsulated components that manage their own state, then combine them to create complex user interfaces. Because component logic is written in JavaScript rather than templates, it is simple to pass rich data through your app while keeping state out of the DOM.[16]

3.3 Conclusion

This chapter was dedicated to fill in the the blanks about the system design. An abstract level explaining of each technology, service and framework was set in place in order to make it easier to grasp the experimental protocols in the next chapter.

CHAPTER 4
EXPERIMENTAL PROTOCOLS

4.1 Introduction

In this chapter, I detail the experimental protocols needed for the the establishment of the network monitoring environment.

4.2 Working environment

- A virtual environment will be running a CentOS Linux 8 minimal operating system.
- Kernel 4.18.0-240.1.1.e18_3.x86_64 on a an x86_64
- VmWare as a virtualisation software.

4.2.1 Settings

These settings are the settings I chose to work with during this project, these settings can be changed depending on the client's preferences and hardware compatibility.

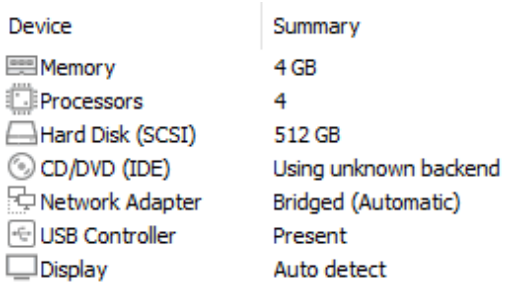


FIGURE 4.1 CentOS 8 VM settings

4.2.2 Machine's initial configuration

1. Download and open CentOS 8 virtual machine in VmWare.
   Download link : `https://techloudgeek.com/download/image/?link=https://dldatac.linuxvmimages.com/VMware/C/8/CentOS_8.3.2011_VMM.zip`
2. After remote logging to the machine with SSH protocol, I need to configure a static

IP address for this machine. To open network manager tool, run

```
$ sudo nmtui
```

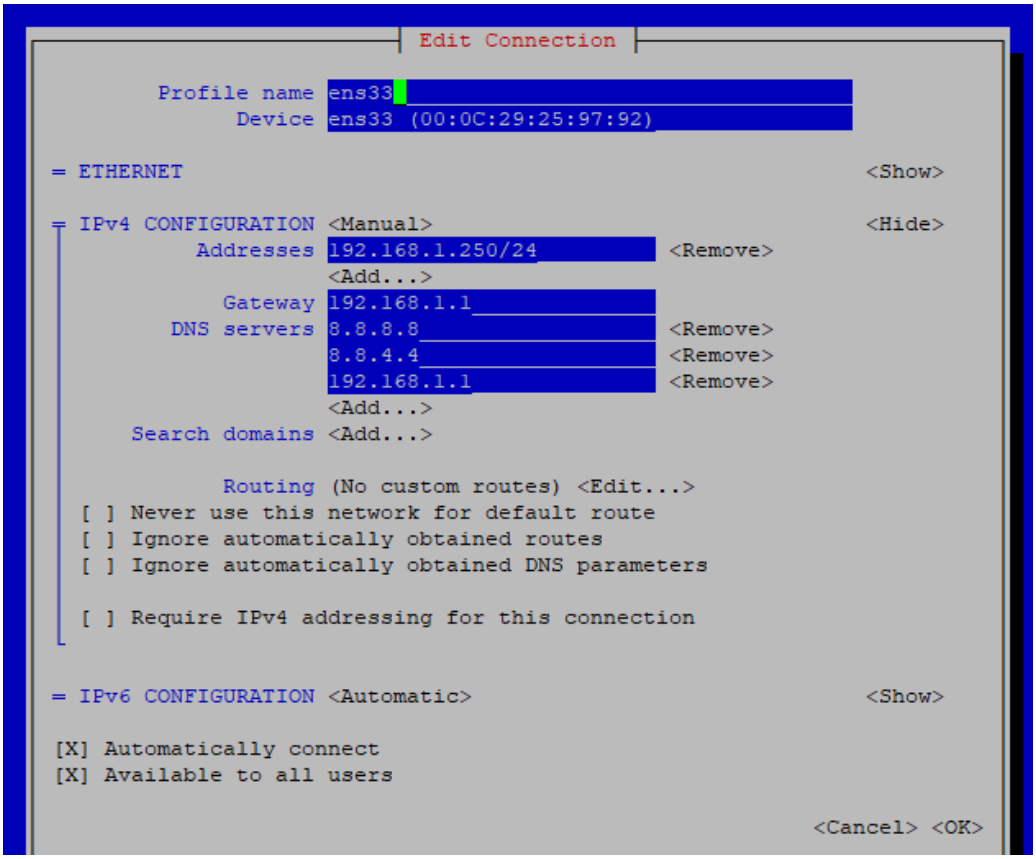


FIGURE 4.2 Server IP address configuration

I chose and configured the desired static IP address **192.168.1.250** using the "Edit a connection" option, then deactivate and reactivate the network interface to see the changes take effect.

3. Disable firewall by running

```
$ sudo systemctl disable firewalld
```

4. To set selinux mode to permissive, run this command to edit the file and change the SELINUX variable to permissive,

```
$ nano /etc/selinux/config
```

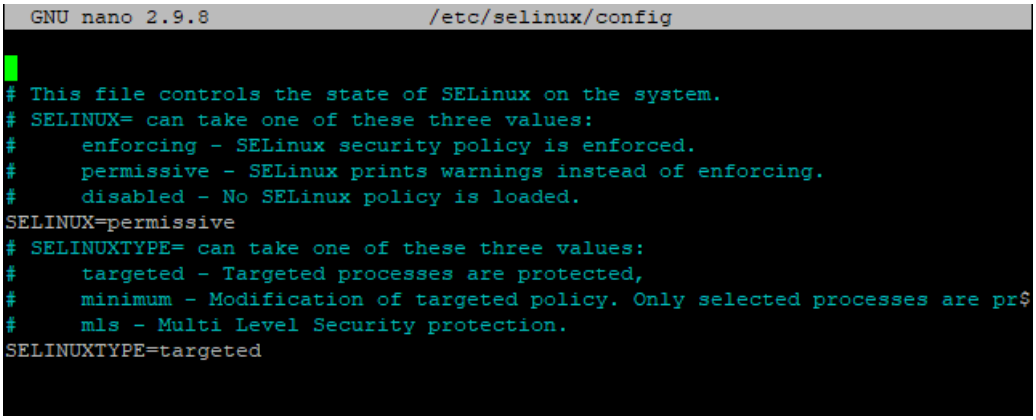


FIGURE 4.3 SELINUX configuration

4.3 Nginx installation

1. Install nginx and it dependencies with the command

   ```
   $ sudo dnf install nginx
   ```

2. Enable nginx daemon starting when the machine starts with the command

   ```
   $ sudo systemctl enable nginx
   ```

3. Start Nginx with the command

   ```
   $ sudo systemctl start nginx
   ```

4. Verify installation and activation succes

   ```
   $ sudo systemctl status nginx
   ```

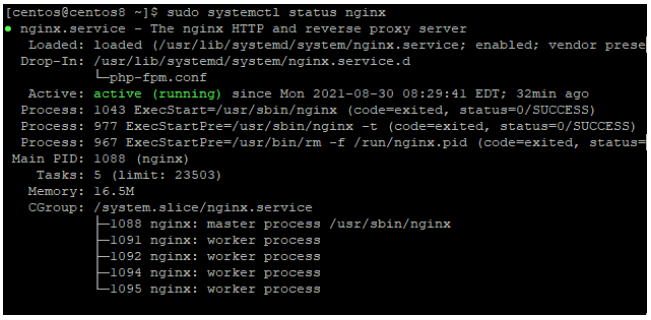


FIGURE 4.4 Nginx service active

4.4 MySQL installation

1. To install MySQL server and its dependencies, run

```
$ sudo dnf install mysql-server
```

2. Enable MySQL server daemon upon start with

```
$ sudo systemctl enable mysqld
```

3. Start MySQL with the command

```
$ sudo systemctl start mysqld.service
```

4. Set MySQL root password and initial configuration options with this command

```
$ sudo mysql_secure_installation
```

4.5 Zabbix Installation

1. Zabbix repository installation

```
$ sudo rpm -Uvh https://repo.zabbix.com/zabbix/5.0/rhel/8
  /x86_64/zabbix-release-5.0-1.el8.noarch.rpm
$ sudo dnf clean all
$ sudo dnf install zabbix-server-mysql zabbix-Ib-mysql
  zabbix-nginx-conf zabbix-agent
```

2. Create initial database for zabbix

```
$ sudo mysql -u root -p <rootPassword>
  mysql> create database zabbix character set utf8
  collate utf8_bin;
  mysql> create user zabbix@localhost identified
  by 'zabbixPassword';
```

```
mysql> grant all privileges on zabbix.*
to zabbix@localhost;
mysql> quit;
```

These lines of mysql commands will create a zabbix database, along with a user that I'll name 'zabbix' that has his own mysql password 'zabbixPassword'. This user is granted all privileges on the Zabbix database.

3. Now I populate the database with the initial Zabbix data tables that comes in a compressed file called create.sql.gz

```
# zcat /usr/share/doc/zabbix-server-mysql*/create.sql.gz
  | mysql -u zabbix -p zabbixPassword zabbix
```

4. Configure the database for the Zabbix server by editing the configuration file zabbix_server.conf. Uncomment and change the value of DBPassword variable 'zabbixPassword' (our zabbix database password).

```
$ sudo nano /etc/zabbix/zabbix_server.conf
```

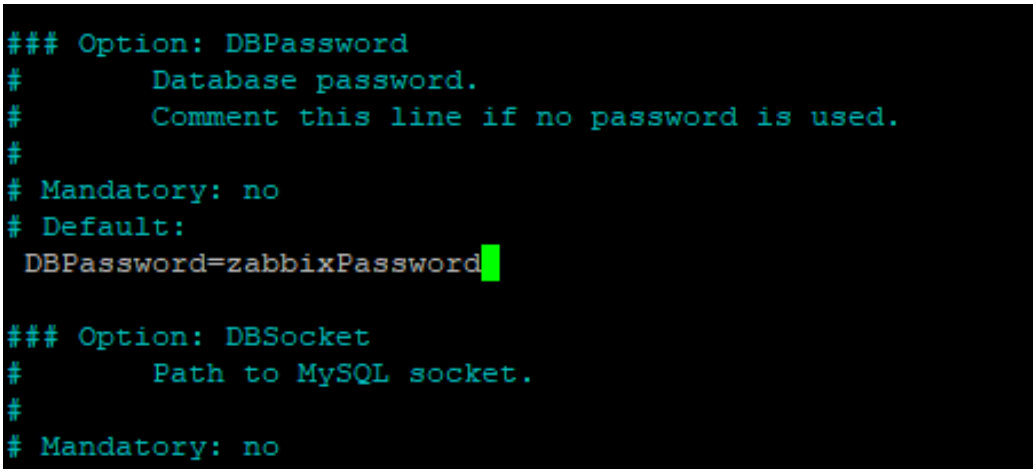


FIGURE 4.5 Zabbix server password configuration

5. Configure Nginx and PHP for the zabbix GUI
   - Uncomment and set 'listen' variable to port 80 and 'server_name' to 'example.com' in the zabbix reverse proxy Nginx file called 'zabbix.conf'

```
$ sudo nano /etc/nginx/conf.d/zabbix.conf
```

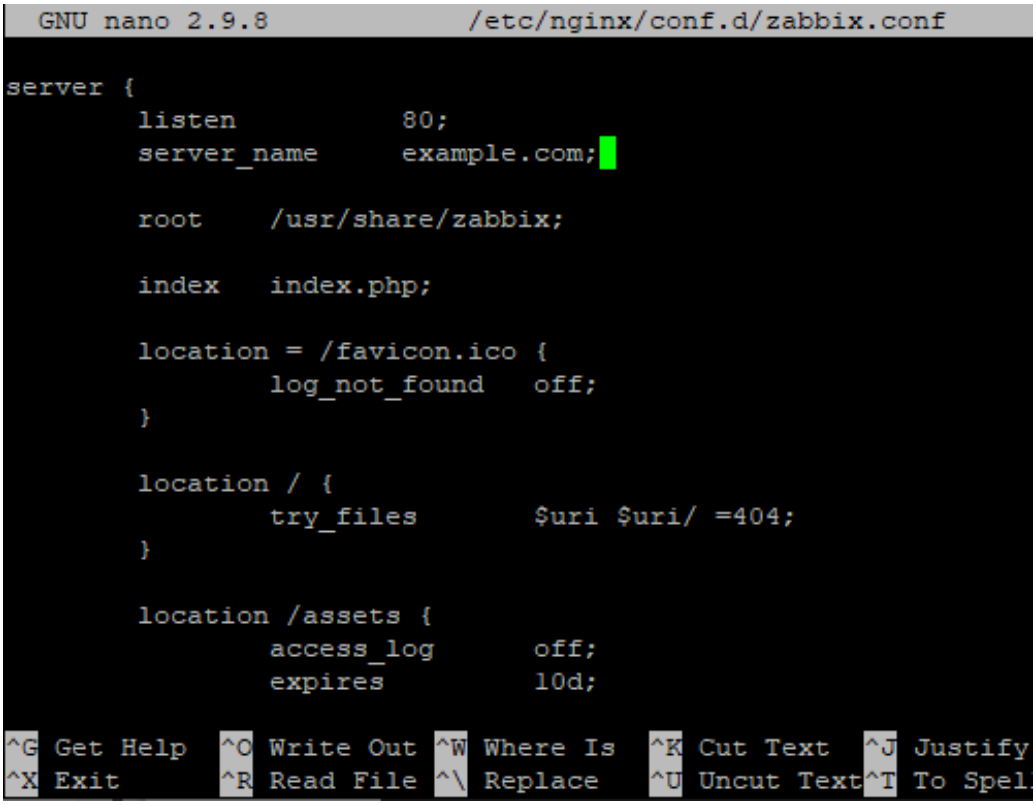


FIGURE 4.6 Zabbix reverse proxy configuration

- Uncomment and set the right timezone for the php-fpm service. Change the php_value[date.timezone] variable to the right time zone value.

  ```
  $ sudo nano /etc/php-fpm.d/zabbix.conf
  ```

```
  GNU nano 2.9.8                /etc/php-fpm.d/zabbix.conf

pm.start_servers = 5
pm.min_spare_servers = 5
pm.max_spare_servers = 35

php_value[session.save_handler] = files
php_value[session.save_path]    = /var/lib/php/session

php_value[max_execution_time] = 300
php_value[memory_limit] = 128M
php_value[post_max_size] = 16M
php_value[upload_max_filesize] = 2M
php_value[max_input_time] = 300
php_value[max_input_vars] = 10000
php_value[date.timezone] = Africa/Tunis
```

FIGURE 4.7 Zabbix-PHP timezone configuration

6. Finally I start Zabbix server and agent processes

```
$ sudo systemctl restart zabbix-server zabbix-agent
  nginx php-fpm
$ sudo systemctl enable zabbix-server zabbix-agent
  nginx php-fpm
```

Now I can access zabbix GUI through the http port 80, by typing the machine IP address 192.168.1.250 as our url.
Zabbix login credentials are the username 'Admin' and the password 'zabbix'.

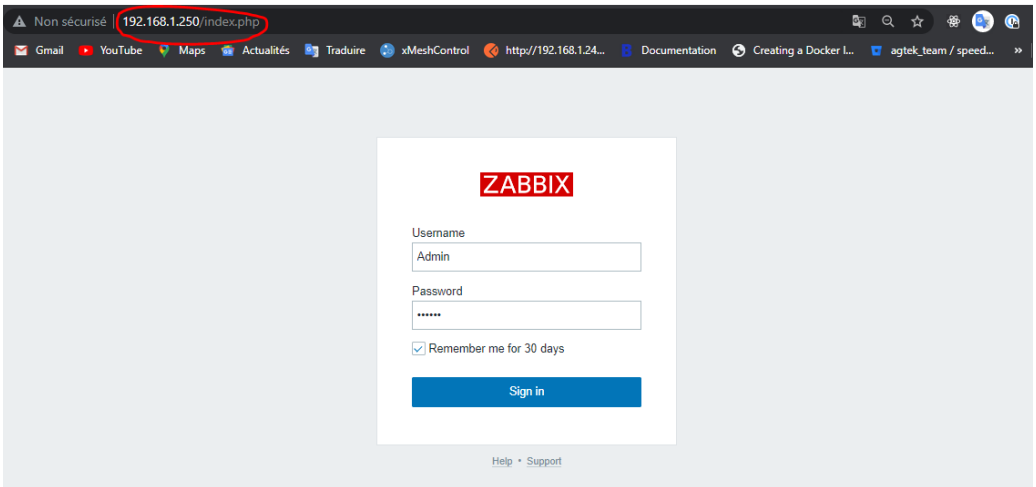


FIGURE 4.8 Zabbix login screen

4.6 Grafana installation

1. Add Grafana repository and GPG key

```
$ cat <<EOF | sudo tee /etc/yum.repos.d/grafana.repo
  [grafana]
  name=grafana
  baseurl=https://packages.grafana.com/oss/rpm
  repo_gpgcheck=1
  enabled=1
  gpgcheck=1
  gpgkey=https://packages.grafana.com/gpg.key
  sslverify=1
  sslcacert=/etc/pki/tls/certs/ca-bundle.crt
  EOF
```

2. Install Grafana and its dependencies

```
$ sudo dnf -y install grafana
```

3. Start and enable grafana-server daemon upon machine start

```
$ sudo systemctl enable --now grafana-server.service
```

4. Creation of Grafana's reverse proxy with Nginx:
   I wrote a custom Nginx configuration script for Grafana reverse proxy, the Grafana web interface will be served as a proxy pass from listening at port 3000 'localhost:3000' to port 8080 'localhost:8080/grafana/', and the script sets the necessary configurations in order to handle the requests and responses headers and options. This is practical for handling the Grafana APIs that I'll use later on.

```
$ sudo nano /etc/nginx/conf.d/grafana.conf
```

```
server {
```

```
    listen 8080;
    root /usr/share/nginx/www;
    index index.html index.htm;

    location /grafana/ {
     proxy_pass http://localhost:3000/;
  if ($request_method = 'OPTIONS') {
          add_header 'Access-Control-Allow-Origin' '*';
          add_header 'Access-Control-Allow-Methods' 'GET,
          POST,
          OPTIONS';

          add_header 'Access-Control-Allow-Headers'
          'Origin,
          Authorization, Accept, DNT,User-Agent,
          X-Requested-With,
          If-Modified-Since,Cache-Control,Content-Type,
          Range';
       }
       if ($request_method = 'POST') {
          add_header 'Access-Control-Allow-Origin' '*';
          add_header 'Access-Control-Allow-Methods' 'GET,
          POST,
          OPTIONS';
          add_header 'Access-Control-Allow-Headers'
          'Origin,
          Authorization,
          Accept,DNT,User-Agent,X-Requested-With,
          If-Modified-Since,Cache-Control,Content-Type,
          Range';
          add_header 'Access-Control-Expose-Headers'
          'Content-Length,
```

```
            Content-Range';
        }
        if ($request_method = 'GET') {
            add_header 'Access-Control-Allow-Origin' '*';
            add_header 'Access-Control-Allow-Methods' 'GET,
            POST,
            OPTIONS';
            add_header 'Access-Control-Allow-Headers'
            'Origin,
            Authorization,
            Accept,DNT,User-Agent,X-Requested-With,
            If-Modified-Since,Cache-Control,Content-Type,
            Range';
            add_header 'Access-Control-Expose-Headers'
            'Content-Length,
            Content-Range';
        }
    }
}
```

5. Edit Grafana configuration file called grafana.ini

```
$ sudo nano /etc/grafana/grafana.ini
```

I need to change these variables in the Grafana configuration file in order for Nginx to properly serve Grafana's web interface.

```
domain = localhost

root_url = %(protocol)s://%(domain)s:%(port)s/grafana/

serve_from_sub_path = true
```

```
max_open_conn = 1000

allow_embedding = true

min_refresh_interval = 1s

enabled = true
```

6. I can now access Grafana's web interface by visiting the url below, and login with the username 'admin' and the password 'admin'
   http://192.168.1.250:8080/grafana

4.7 Automation of Zabbix monitoring

Now that I have Zabbix set in place and ready to be used, it's time to properly configure and automate different entities in order to make it discover Xtremesh nodes in the network without manually adding them each time, as well as automating the creation of the applications and the items that handles the collection of metrics in each new host discovered. This process of automation requires that I create a template for the SNMP monitoring of these devices, import it to Zabbix, then set up a low level discovery rule (LLD rule) where it will check some conditions that I specify. If these conditions are met and the rule verifies that the discovered device is an Xtremesh node, it adds it to the group of the the already discovered and configured Xtremesh nodes and applies the template to it. This process ensures that Zabbix adds any new Xtremesh node connected from the network to be monitored automatically.

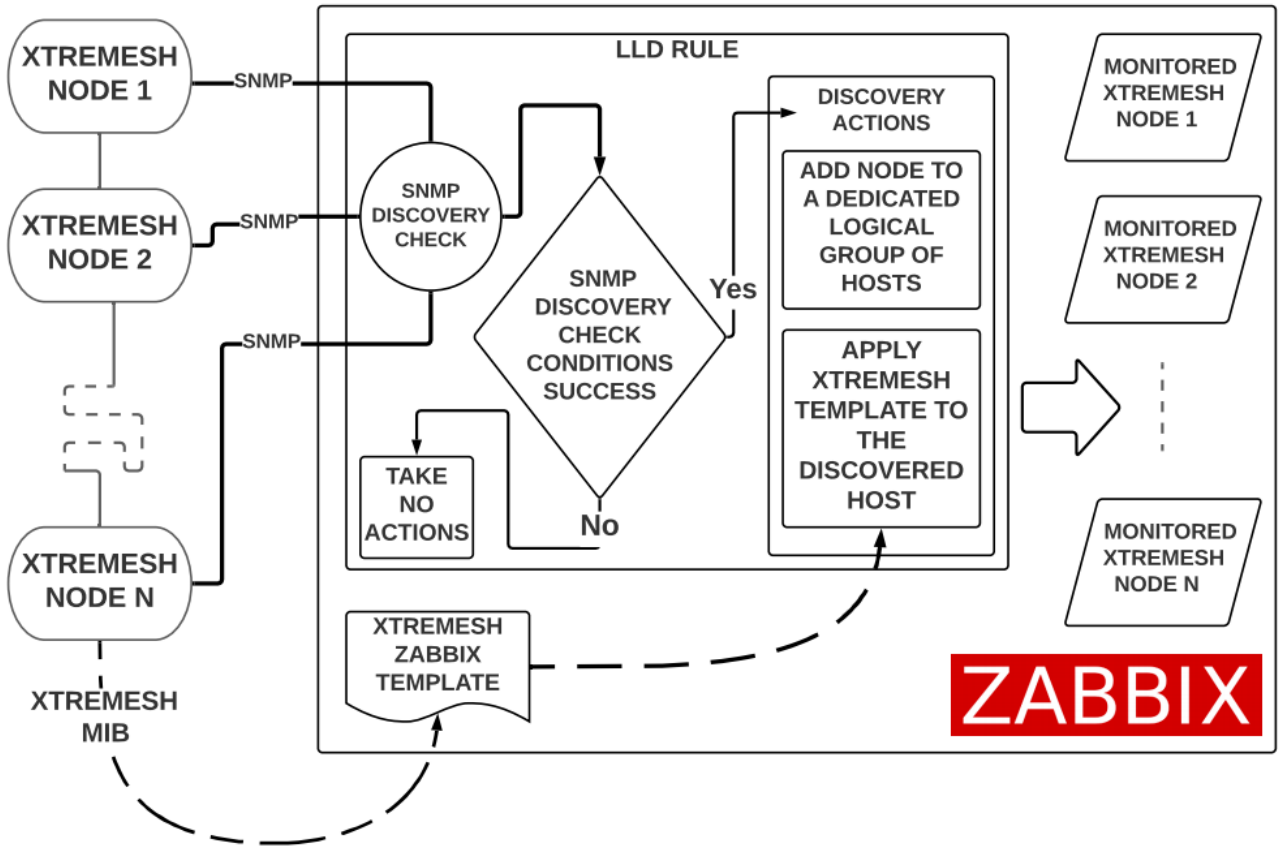


FIGURE 4.9 Zabbix automation flow

4.7.1 Creation of Xtremesh's Zabbix template

The use of templates is an excellent way to make Zabbix monitoring much easier. A template can be assigned a set of entities, allowing for the efficient management of thousands of devices.

All of the items in an Xtremesh node that needs to be monitored will be created using an XML template that defines the SNMP data to be collected. the OIDs stored in each host's MIB will be used to do so.
The template is to be applied automatically to each discovered node through a predefined process called low level discovery action.
Here's a briefing to understand what I'll do :

- Each Xtremesh node has SNMP protocol enabled.
- A MIB, that is a database used for managing the entities in a communication network (associated with the snmp protocol), is already defined in each device.

- A MIB file should contain all the necessary OIDs (object id) of each metric to be collected.
- Using this MIB file, I can create a Zabbix template defining all items informations that needs to be used to collect these metrics.

First of all, I need to easily read the Xtremesh MIB file, any MIB browsing tool should do the job. Here I'll be using iReasoning MIB browser software.
Download link: `https://www.ireasoning.com/download/mibfree/setup.exe`

1. Go to FILE then click on Load MIBs option, and choose CODINTEK-MIB (which is the name of the MIB file provided by CodinTek Engineers that contains the OIDs to be used)
2. I can now browse all the data provided from the MIB file. This figure shows the data pulled from the Xtremesh device with the IP address of **192.168.1.138** via SNMP protocol, with the request method: GET SNMP <OID>. The OID of the device model object is ".1.3.6.1.4.1.8888.1.1.1.0", and its value is the string "XTM-200". Each object has an OID, and I need to use these OIDs to tell zabbix what data to be collected from the devices. These OIDs then will be hardcoded with XML in a special structured template file. So that every time a new device is discovered in the network, Zabbix applies this template to automatically pull the data from these devices.

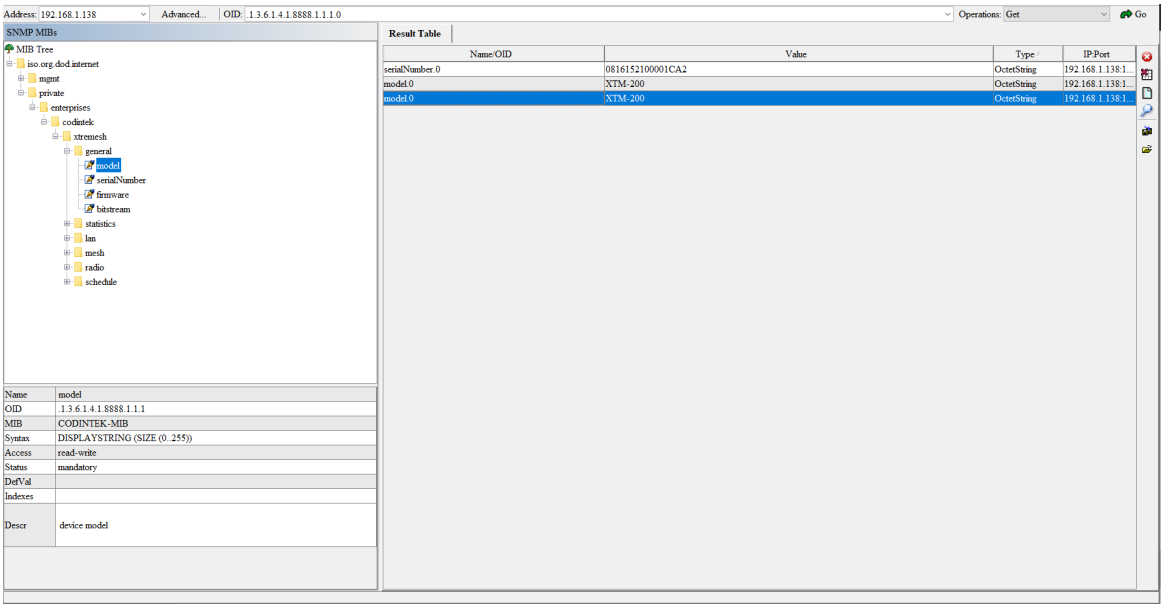


FIGURE 4.10 Xtremesh MIB tree

3. Right now I need to take note each OID and its descriptive name, and organize these data to use them later to create the Zabbix template.

   I chose to organize the data in a JSON format file for a cleaner reading and classification.

   This figure below shows a sample of the data that I extracted from the MIB browser. The complete file contains a set of 89 labeled OIDs.

mib.json - Bloc-notes

Fichier Edition Format Affichage Aide

```
{
      "general" : {
            "model" : ".1.3.6.1.4.1.8888.1.1.1.0",
            "serialNumber" : ".1.3.6.1.4.1.8888.1.1.2.0",
            "firmware" : ".1.3.6.1.4.1.8888.1.1.3.0",
            "bitstream" : ".1.3.6.1.4.1.8888.1.1.4.0"
      },

      "statistics" : {
            "RfInterfaces" : {
                  "UpInterfaces" : {
                        "neighIndex" : ".1.3.6.1.4.1.8888.1.2.1.1.1.0",
                        "myAntenna" : ".1.3.6.1.4.1.8888.1.2.1.1.2.0",
                        "neighAntenna" : ".1.3.6.1.4.1.8888.1.2.1.1.3.0",
                        "usedTxPower" : ".1.3.6.1.4.1.8888.1.2.1.1.4.0",
                        "refTxSnr" : ".1.3.6.1.4.1.8888.1.2.1.1.5.0",
                        "lastTxSnr" : ".1.3.6.1.4.1.8888.1.2.1.1.6.0",
                        "refRxSnr" : ".1.3.6.1.4.1.8888.1.2.1.1.7.0",
                        "lastRxSnr" : ".1.3.6.1.4.1.8888.1.2.1.1.8.0",
                        "upRxRSSI" : ".1.3.6.1.4.1.8888.1.2.1.1.9.0"
                  },
                  "DownInterfaces" : {
                        "neighIndex" : ".1.3.6.1.4.1.8888.1.2.1.2.1.0",
                        "myAntenna" : ".1.3.6.1.4.1.8888.1.2.1.2.2.0",
                        "neighAntenna" : ".1.3.6.1.4.1.8888.1.2.1.2.3.0",
                        "usedTxPower" : ".1.3.6.1.4.1.8888.1.2.1.2.4.0",
                        "refTxSnr" : ".1.3.6.1.4.1.8888.1.2.1.2.5.0",
                        "lastTxSnr" : ".1.3.6.1.4.1.8888.1.2.1.2.6.0",
                        "refRxSnr" : ".1.3.6.1.4.1.8888.1.2.1.2.7.0",
                        "lastRxSnr" : ".1.3.6.1.4.1.8888.1.2.1.2.8.0",
                        "dwRxRSSI" : ".1.3.6.1.4.1.8888.1.2.1.2.9.0"
                  }
            },

            "WirelessInterfaces" : {
                  "UpWireless" : {
                        "upSentBeaconsNbr" : ".1.3.6.1.4.1.8888.1.2.2.1.1.0",
                        "upSentMPDUsNbr" : ".1.3.6.1.4.1.8888.1.2.2.1.2.0",
```

FIGURE 4.11 MIB's structured data sample

4. Now that everything is in order and the data is well structured, I can now start writing the Zabbix template. A basic XML structure of the template should look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<zabbix_export>
    <version>ZABBIX_VERSION</version>
    <date>DATE</date>

    <groups>
        //Definitions of zabbix groups
        //to use in this template
    </groups>

    <templates>
        <template>
```

```
            <template>NAME_OF_THE_TEMPLATE</template>
            <description>
                //brief description of the template
            </description>
            <groups>
                ...
            </groups>

            <applications>
                ...
            </applications>

            <items>
                ...
            </items>
        </template>
    </templates>
</zabbix_export>
```

5. This below is a snippet of the XML code of the template.

```
<?xml version="1.0" encoding="UTF-8"?>
<zabbix_export>
    <version>5.0</version>
    <date>2021-05-25T15:40:46Z</date>
    <groups>
        <group>
            <name>Templates/Network devices</name>
        </group>
    </groups>
    <templates>
        <template>
            <template>codintek-mib</template>
```

```
<name>codintek-mib</name>
<description>
    //Template for codintek's
    //Xtremesh devices
</description>
<groups>
    <group>
        <name>
            Templates/Network devices
        </name>
    </group>
</groups>
<applications>
    .
    .
    <application>
        <name>general</name>
    </application>
    .
    .
</applications>
<items>
    .
    .
    <item>
        <name>switchInstant</name>
        <type>SNMP_AGENT</type>
        <snmp_oid>
            .1.3.6.1.4.1.8888.1.6.1.0
        </snmp_oid>
        <key>switchInstant</key>
        <delay>1s</delay>
```

```
                    <description>
                        switchInstant
                    </description>
                    <applications>
                        <application>
                            <name>
                                schedule
                            </name>
                        </application>
                    </applications>
                </item>
                 .
                 .
            </items>
        </template>
    </templates>
</zabbix_export>
```

For a simple show case, in the XML code above I only showed how to add one item called 'switchInstant' with the OID '.1.3.6.1.4.1.8888.1.6.1.0' that belongs to a Zabbix application called 'general'. The completed template contains 89 items and 13 applications. Each application groups a number of correlated items. After importing this file to Zabbix, it will take the name of "codintek-mib" as specified in the name section of the template.

4.7.2 Importing the template to Zabbix

When I navigate to the templates section of Zabbix, I find a large list of already predefined templates for almost every type of device known in the market, configured and maintained by the device's manufacturer.

1. In the zabbix GUI, go to Configuration —>Templates. Many templates are already listed.

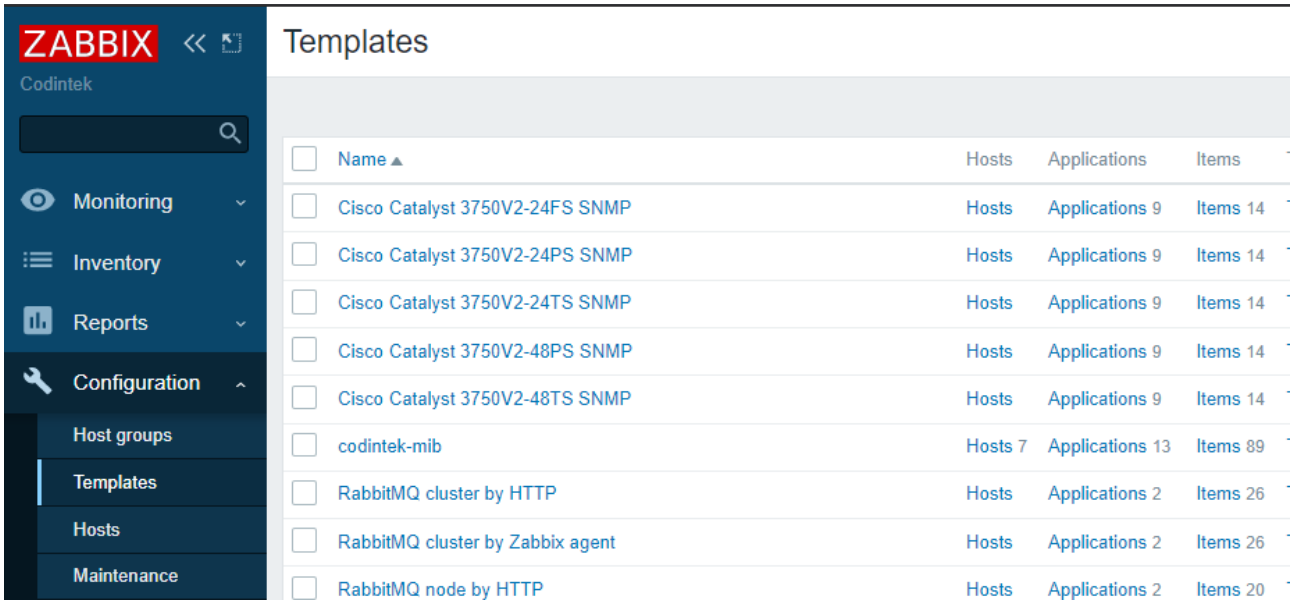


FIGURE 4.12 Zabbix's templates list

2. I need to import my own template that I created earlier so I can use it in the low level discovery action process. Go to the top right side of the page and click on 'import', then choose the XML template file created.

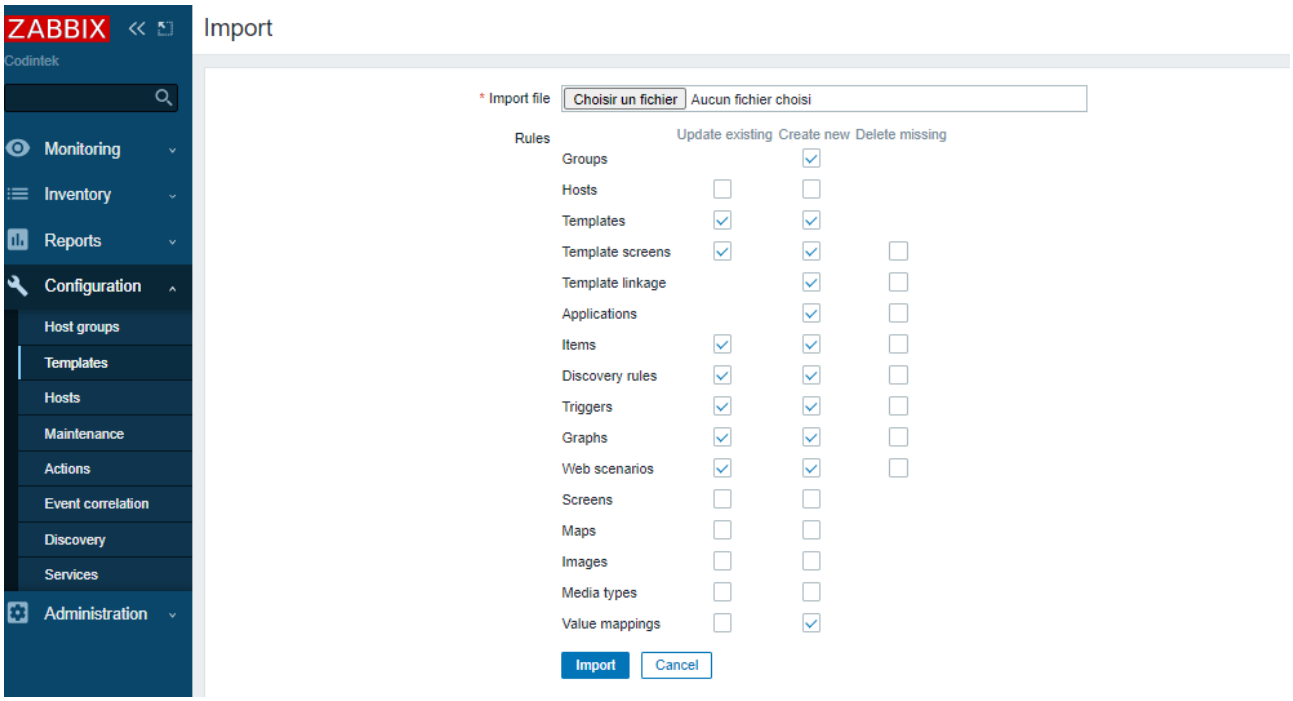


FIGURE 4.13 Import Template to Zabbix

3. After successful importation, I can see that my template, called codintek-mib, is present along with the other templates and ready to be used.



FIGURE 4.14 codintek-mib successful import

4.7.3 Zabbix LLD rule (Low Level Discovery)

Low-level discovery allows to automatically generate items, triggers, and graphs for various entities. Zabbix can automatically begin monitoring new network devices and interfaces, eliminating the need to manually create items for each network interface. The items that need to be monitored on each new device discovered in the network are defined in the codintek-mib template.

Steps:

- I configure a discovery rule that performs a SNMP check on new devices in the network and determines whether the device is an Xtremesh node by checking if the OID labeled sys.description or "1.3.6.1.2.1.1.1.0", contains the string value "lwIp" (lwIp acts as the Xtremesh device identifier).
- If this condition is met, I can be certain that I discovered a new Xtremesh node, so I set discovery actions to tell Zabbix to add this device to a specific group of hosts, which I'll call xmeshGroup, and to automatically assign the template I created to each one of the discovered nodes.

This process should eliminate the work of manual configuration of each host individually, especially in this case where there could be hundreds of Xtremesh nodes, implying the labour configuration of thousands of items. So let's now begin setting up the LLD rule.

1. In this project, for the sake of optimisation the process of scanning the network and configuring each node, which takes a long time depending on the number of items and devices found, I have set up 9 discovery rules instead of only one, each rule scans a small IP range containing approximately 30 IP addresses each. This ensures parallel processing threads rather than one single thread, which greatly reduces processing time.
   (a) Go to Configuration —> Discovery —> click on Create discovery rule button.
   (b) I will now configure the discovery rule check of the first IP range, named

xmesh1-30 which scans the range from 192.168.1.1→192.168.1.30 (30 IP addresses), and checks SNMP availability on the scanned devices.

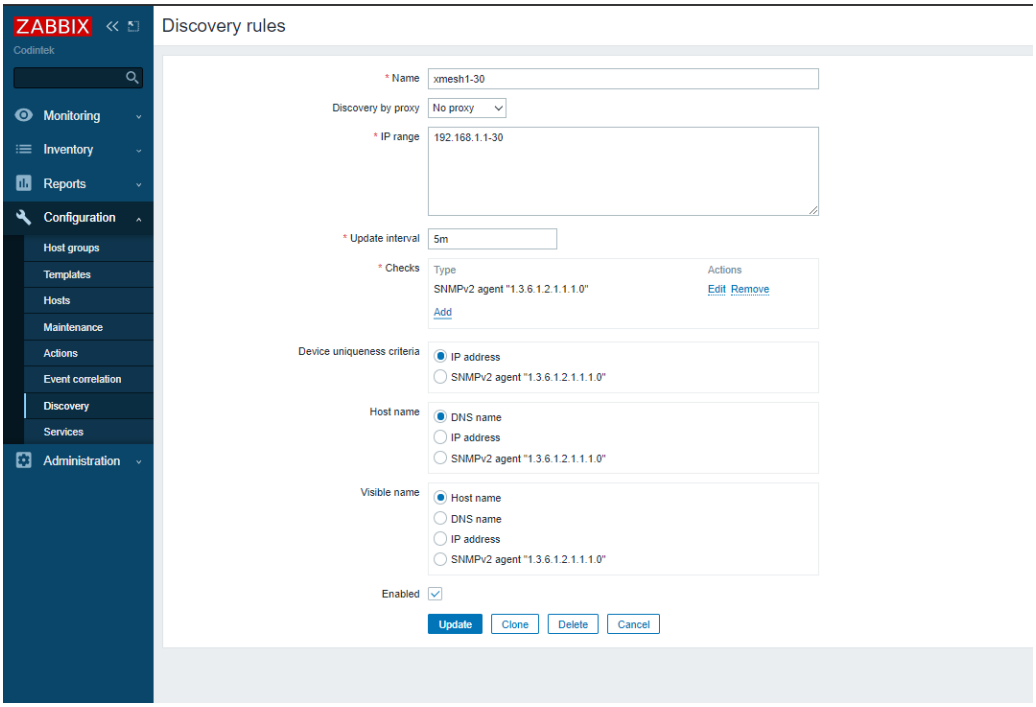


FIGURE 4.15 Discovery check configuration

The 'Checks' field is for the type of the checks that I want to add . Click on add in the Checks field and fill the form for an SNMPv2 check that verifies the availability of SNMP by querying the sys.Description OID.

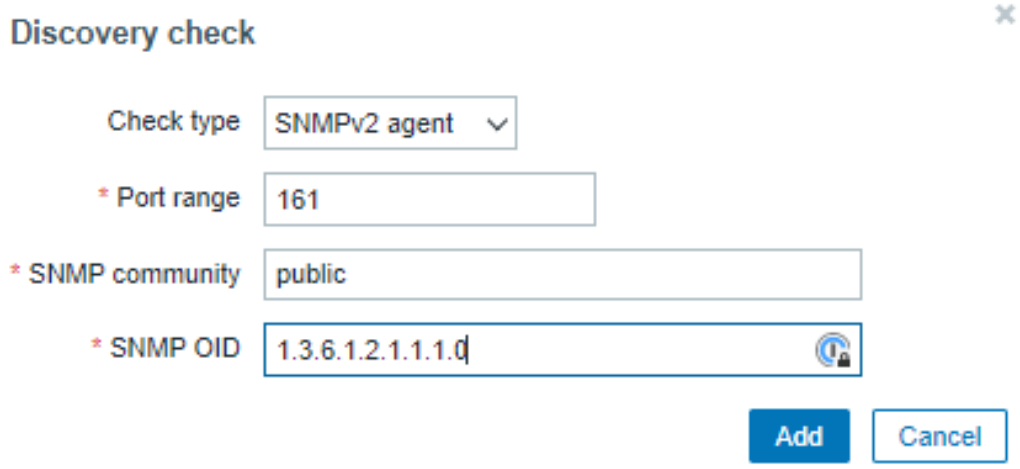


FIGURE 4.16 SNMP OID check

(c) The first discovery rule, which checks the first 30 IP address ranges, is now configured. The same process I had applied to configure the other discovery

rule ranges:
xmesh31-61, xmesh62-92, xmesh93-123, xmesh124-154, xmesh155-185, xmesh186-216, xmesh217-247, xmesh248-254 - xmesh248-254

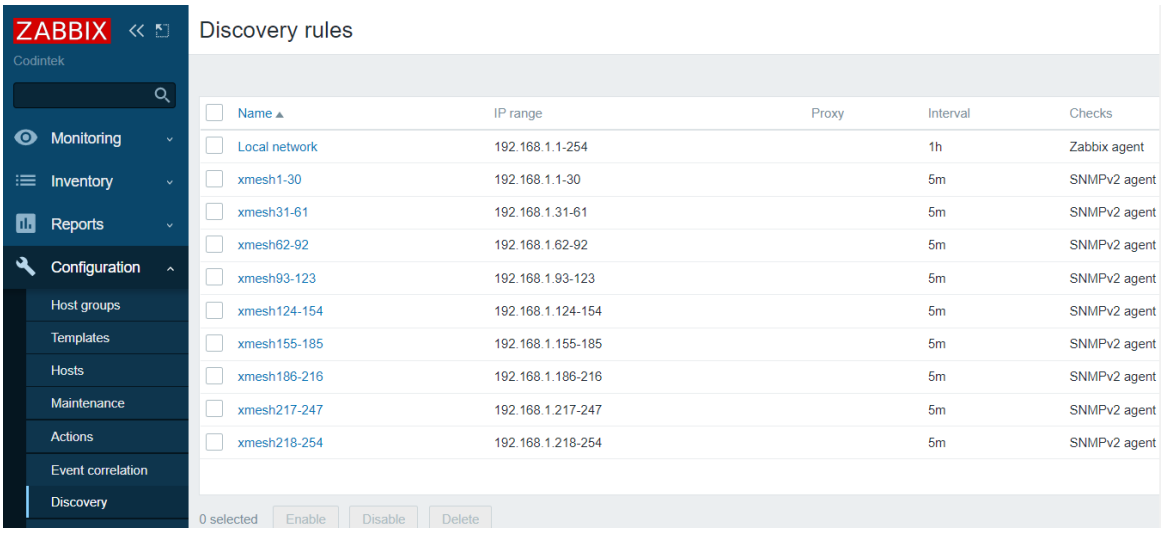


FIGURE 4.17 Discovery's configured SNMP checks

2. After I've set up each discovery rule required to cover 254 IP addresses from 192.168.1.1 to 192.168.1.254, I must now specify the actions that should be taken against the discovered node.
   (a) Go to Configuration → Actions, and select in top left corner the type of action I need, which is a discovery action.

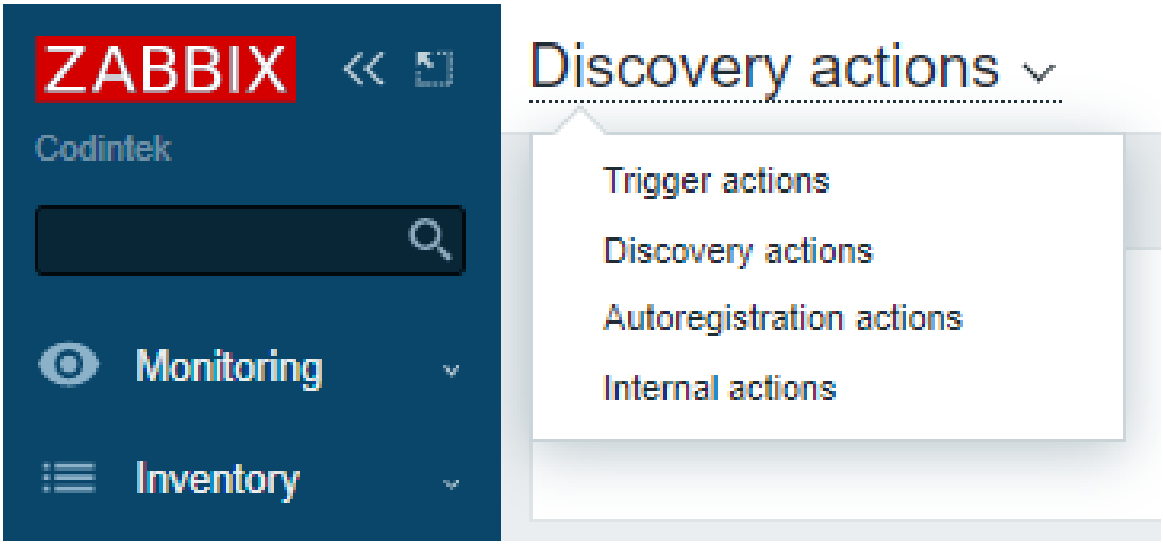


FIGURE 4.18 Discovery Actions drop-down box

   (b) Click on the Create action button in the top right corner, and let's configure

what I need

- I name the action 'xmesh_action'
- I set the conditions for this action's execution:

  - In this action process, the only discovery checks that are valid are the ones I created earlier.

  - The string "lwIp" must be present in the value of the SNMP OID check in these discovery checks, because only Xtremesh devices have the string 'lwIP' as value of sys.description OID.

(c) Select add in the Conditions fields. I need to set up each discovery check until all nine checks (from xmesh1-30 to xmesh248-254) are conditioned. That's how I tell Zabbix to only select the checks I created earlier when applying the discovery actions. This below is an example of filtering the two first checks.

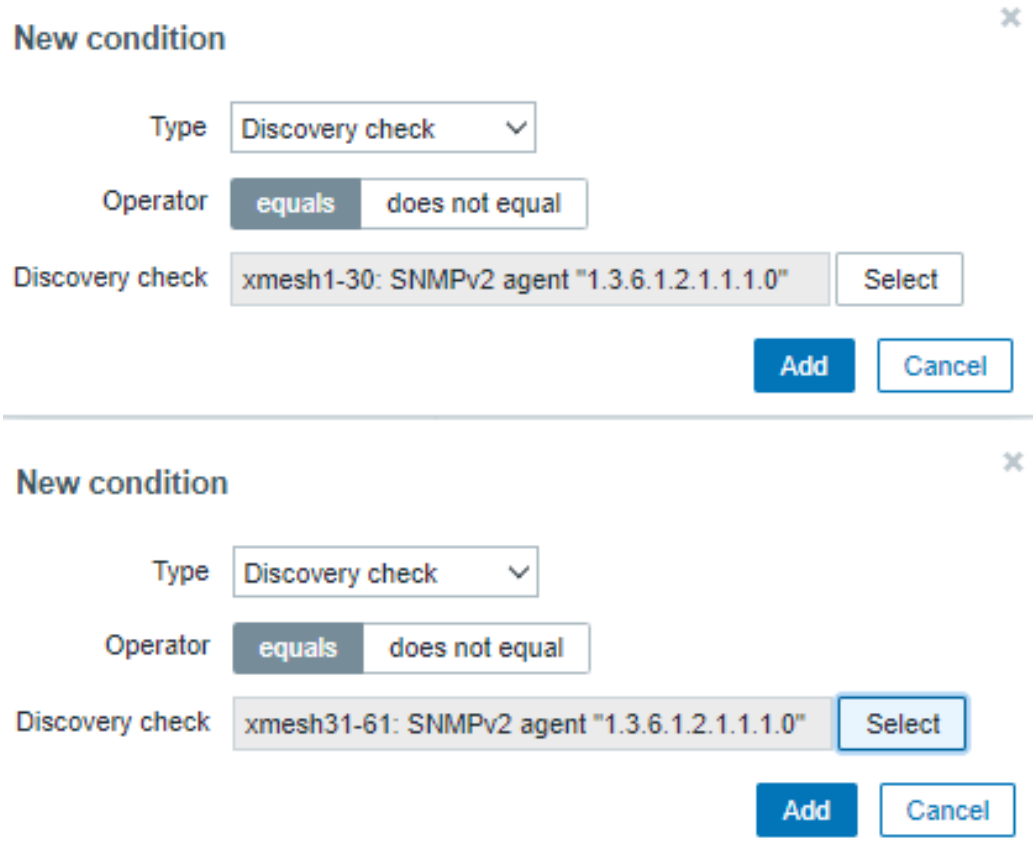


FIGURE 4.19 Conditions to filter only the xmesh discovery checks

Now I must set the other condition, which checks whether the devices discovered have the string value "lwIP" in their sys.description OID.

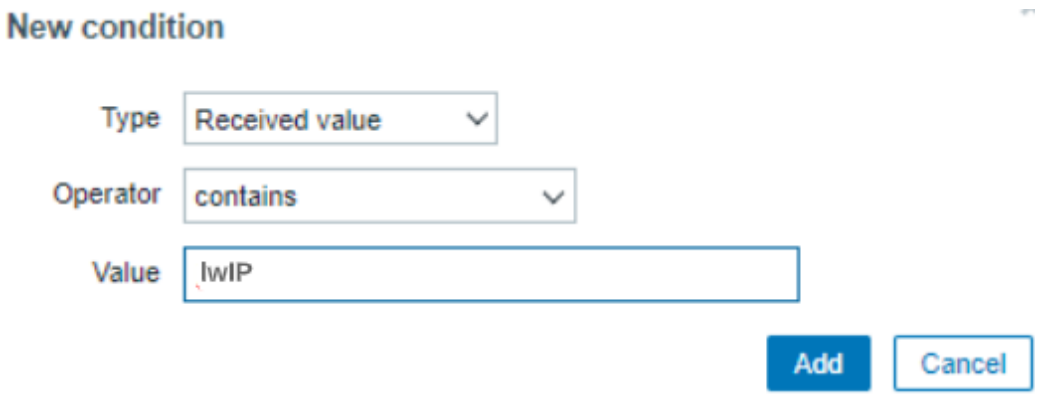


FIGURE 4.20 Condition to filter only the Xtremesh devices

(d) After configuring these conditions, select 'And/Or' in the 'Type of calculation' field. The action section should look like this:

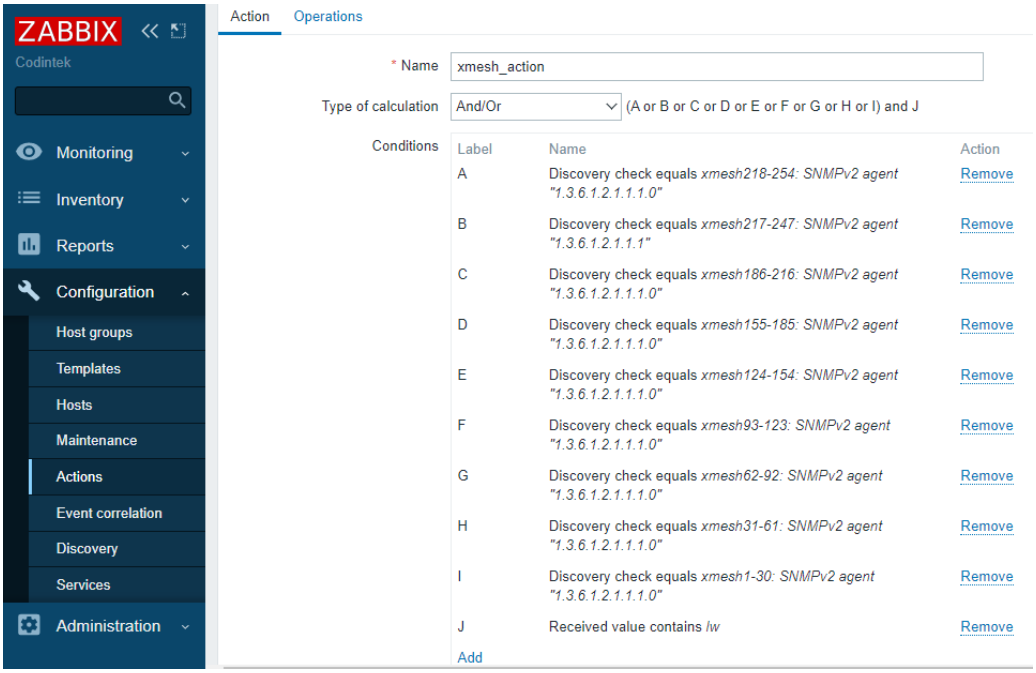


FIGURE 4.21 Action form

Zabbix now tries to verify if one of the 9 discovery checks detects a SNMP device, and if the value contained in the device's sys.Description OID is the string 'lwIP', then Zabbix knows it's a Xtremesh device and will apply some operations to it.

(e) Now that I've completed the conditions for this action to be carried out, all that remains is to specify the operations that will be executed when these conditions are met:

- Add the discovered hosts to the xmeshGroup, which contains all of the nodes that I previously created.
- Assign the codintek-mib template to each discovered node. Go to operations then click on add, Choose the operation type 'Add to host group', then select in the 'Host groups' field 'xmeshGroup' and then click on add.

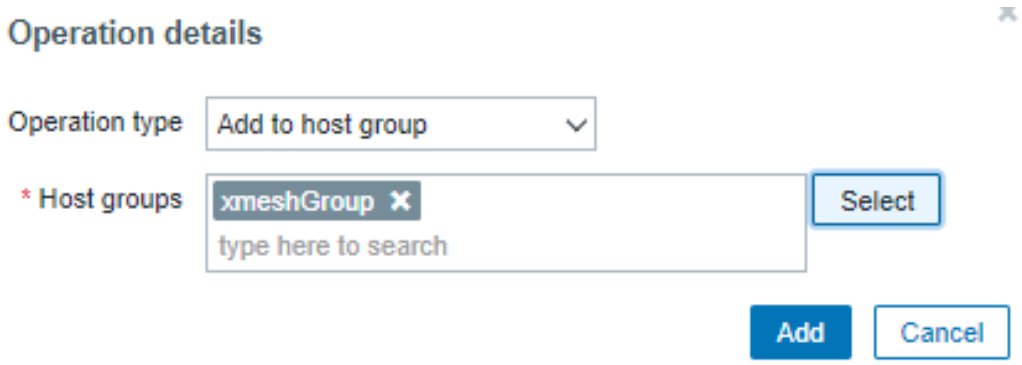


FIGURE 4.22 Add to xmeshGroup operation

Now each discovered Xtremesh node should be placed under the xmeshGroup group of hosts. Now let's add another operation that will assign a template to the discovered node. In the operation type field choose 'Link to template', then in the template field click on select and browse for 'codintek-mib' template, and then click on add.

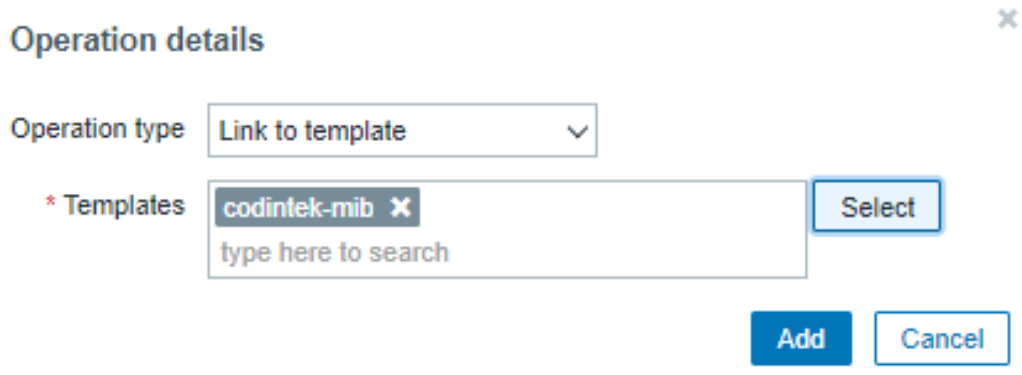


FIGURE 4.23 Link to template operation

The operation field it should like this

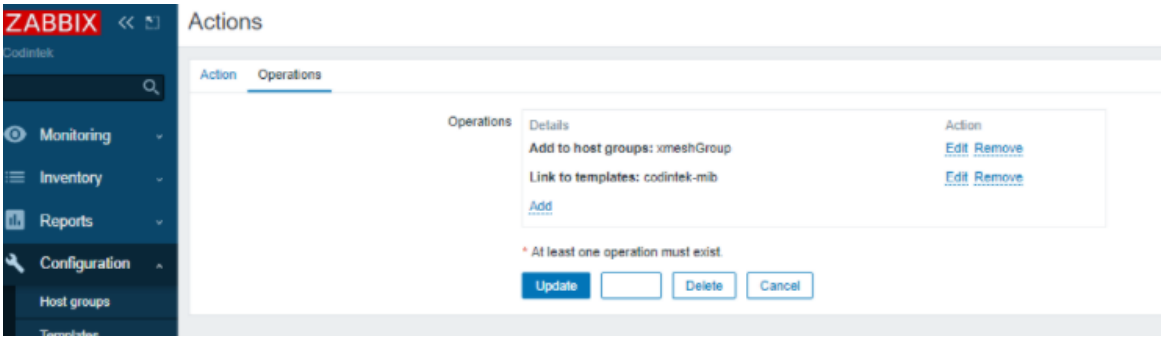


FIGURE 4.24 Operation field

Following these configurations, Zabbix can now detect any new Xtremesh node connected to the network, automatically configure it, and it should be ready to be monitored without having to manually configure anything. In my testing environment, the network has two new devices with IP addresses 192.168.1.137 and 192.168.1.138. Their are automatically discovered, added to xmeshGroup, and linked to a codintek-mib template so that they can be monitored by Zabbix. I can see them in the section Monitoring → Hosts:

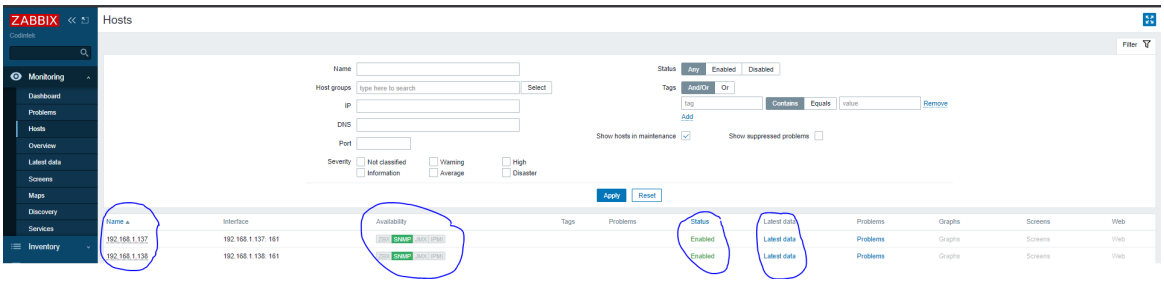


FIGURE 4.25 Monitored Xtremesh hosts

I can see the data being already collected if I click on 'Latest data'.



FIGURE 4.26 Hosts latest data collected

If I navigate to Configuration → Hosts, I can see global statistics about the discovered Xtremesh nodes. It shows that Zabbix is monitoring 89 items in each device. With the LLD process, I only need to start the Zabbix server, and everything should be automated.

4.8 Xtremesh's dashboard model with Grafana

So far, I've been able to automate node discovery and auto data collection using Zabbix. Given that Zabbix has visualisation limitations, having good graphs that render in real time and providing efficient data graphical interpretation is a must.

I will use the grafana service to create graphs, which will be then integrated in the web interface that I will develop. However, the challenge will be in automating the creation of dashboards for each device without human intervention when a node is discovered in Zabbix.

I have now two devices being monitored, with respective IP addresses 192.168.1.137 and 192.168.1.138. My proposed solution is what follows:

- Connect Grafana to Zabbix to be able to pull the data from monitored devices.
- Choose one of devices that are already monitored, and create the graphs needed to visualize its data in a Grafana dashboard.

- Now I have a template dashboard. I want to apply its graphs patterns to each Xtremesh node to be discovered and monitored by Zabbix, that means the same structure of graph panels for each device but the data and variables to configure are different for each node.
- This dashboard that I created can be modeled in JSON code format.

4.8.1 Steps

1. To visualize Zabbix metrics in Grafana, Zabbix plugin for Grafana needs to be installed on the CentOS guest machine. Executing this command will install this plugin:

   ```
   # grafana-cli plugins install alexanderzobnin-zabbix-app
   ```

2. I need to configure Zabbix data source. Go to settings icon on the side bar, then choose plugins option
3. Search for the zabbix plugin that I installed earlier in the search bar, it should appear in the lists of plugins then click on the zabbix plugin and then enable it.
4. After enabling plugin I can add Zabbix data source. To add new Zabbix data source open 'Data Sources' in side panel, click 'Add data source'
5. Select 'Zabbix' from the data sources list and configure data source details

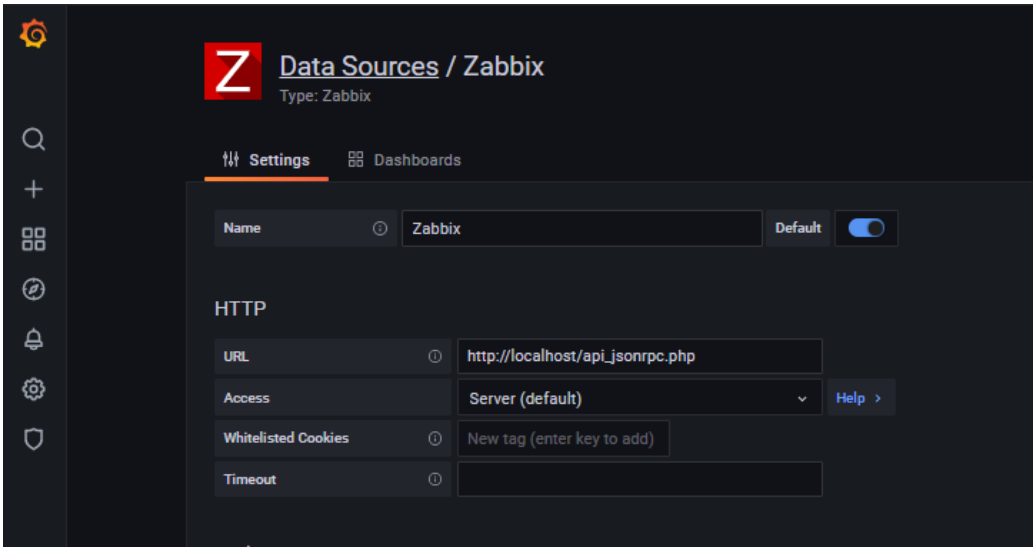


FIGURE 4.27 Zabbix data source configuration

'

6. Then click Save & test. Zabbix data source will now be added.

7. Now that Zabbix is integrated with Grafana, I can use the data collected from Zabbix and display it with Grafana charts.
   I'll create now the first dashboard that displays the metrics collected in real time of the node 192.168.1.137, in the form of time series graphs.
   Lets create the first graphs. Go the + icon on the side bar and click on Create. Choose Add empty panel option, and I'll configure now the query options so that grafana can fetch our data from zabbix and display them

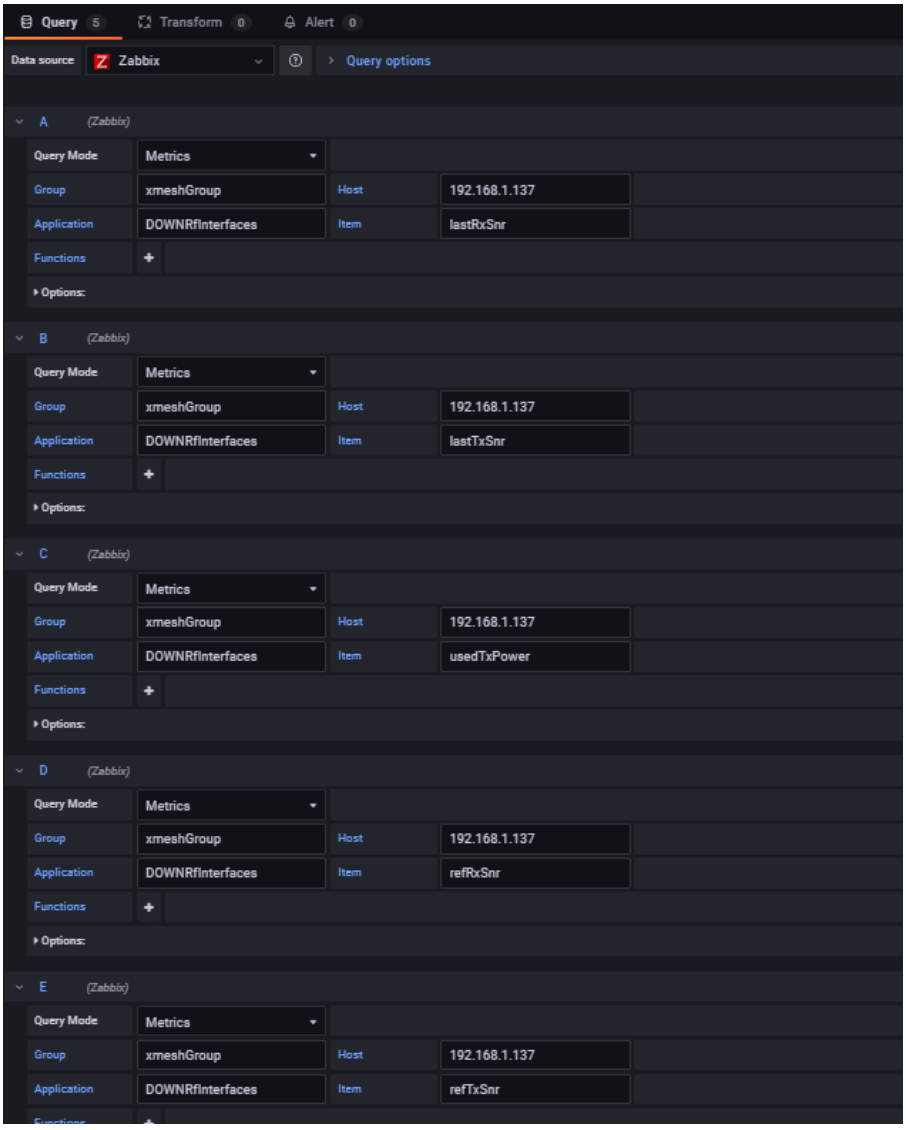


FIGURE 4.28 Graph panel configuration

- Query Mode: Metrics, it means that I'll be only fetching the metrics collected in zabbix and no other data
- Group: xmeshGroup, is the group of nodes I created in zabbix, that contains

only the Xtremesh nodes that are automatically discovered and assigned the codintek-mib template I created in the beginning

- Host: the IP address or name of the Zabbix host
- Application: the application that contains the item that I want to display its data, defined in the codintek-mib template, here I want to select the item contained in application of the downlink RF interfaces
- Item: the item that I want to display its data, here I want to see the last value of SNR (signal noise ratio) when the Xtremesh node is receiving the signal.

8. Each of these queries instructs Grafana to display the item specified in it in the same graph panel as the items specified in the other queries. Now I edit the graph time series parameters so that it looks good to go. Go to the right side and select the top Time series option, then work your way around the panel's styling as you see fit. The name of the panel indicates what I will see in the graph; in this case, I set the name to DOWNSTREAM RF, which means the information I need to see about the use of an RF downstream link.

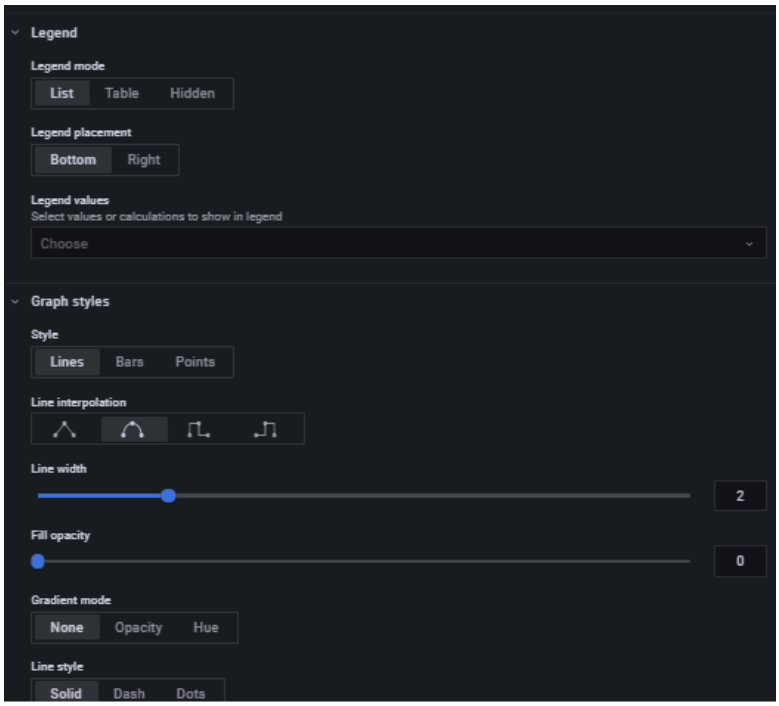


FIGURE 4.29 Graph options and styling

9. The result graph should look like this, Click on Apply to create the panel then save it.



FIGURE 4.30 Down-Stream RF graph for the node 192.168.1.137

10. After I've created all of the necessary graphs, our dashboard for the node 192.138.1.137 should look like this



FIGURE 4.31 192.168.1.137's Dashboard

Because the node 192.1668.1.137 is the root node and only communicates with 192.168.1.138, the UPSTREAM graphs show null data. Meanwhile, 192.168.1.138 communicates only with the root node in the upstream, so when I create its dashboard, the two graphs should show null data in the downstream.

4.8.2 Result

I now have the dashboard ready, but it is only for one node with the IP address 192.168.1.137, and I manually configured all graphs. To automate the process of creating a specific dashboard for a specific node,I need to obtain the dashboard's JSON model and consume both Grafana and Zabbix APIs through JavaScript code that handles the creation of dashboards for each Xtremesh node discovered by Zabbix, then embed the graphs related to each node in the web application.

The JSON model of the dashboard of the Xtremesh node 192.168.1.137 can be obtained directly from Grafana GUI.
Go to dashboard settings then navigate to JSON model section

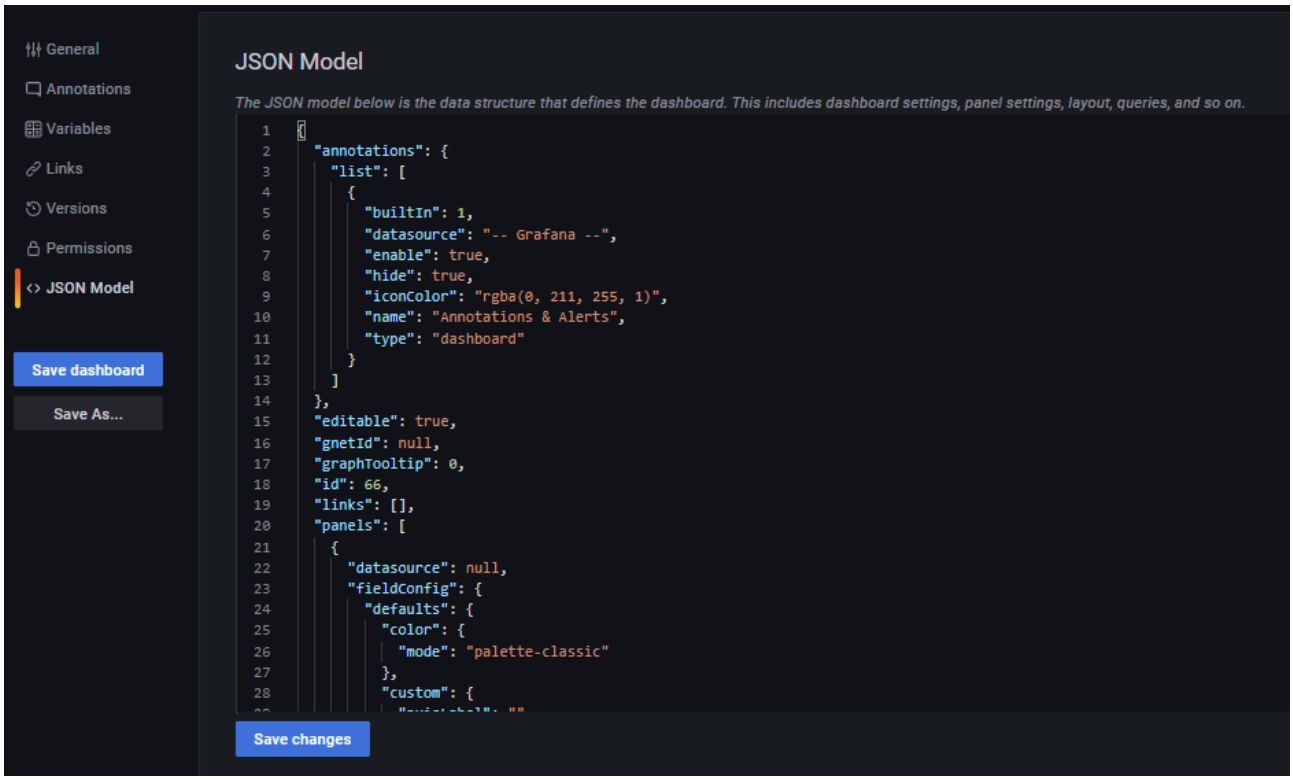


FIGURE 4.32 192.168.1.137's Dashboard Json model

Now everything is in order, Zabbix can automatically begin monitoring any new coming Xtremesh node in the network, and I have prepared the dashboard model in a format of a JSON model, and use that model to automate dashboard creation and embed the graphs in the web platform.

4.9 Conclusion

This chapter is dedicated to serve as a clear picture of the experiment's purpose, the materials required and the major configurations. Now that the monitoring environment is set up, I can proceed to the design and implementation of the Xtremesh web platform.

CHAPTER 5
XTREMESH PLATFORM DEVELOPMENT

5.1 Introduction

Following the phase of setting up the back-end environment (Zabbix server, Grafana server and MySQL database) and its proper configurations, this section details the process of creating an application that serves as a the network management tool for our Xtremesh network.

5.1.1 xMeshControl overview

xMeshControl was the name given to this web application. It's used by the client as the the front view of the NMS system. The following features are required for the platform:

- Dashboard panel for centralized network control and tracking.
- User management system.
- Nodes live interactions and management.
- Changing nodes parameters.

xMeshControl features can be added or changed to meet the needs of the company. This project, however, will only cover for the moment the ones mentioned above.

5.2 Client-server model

The client-server model is a software architecture that divides tasks and workload between a service provider and a service requester. The former is known as the Server, or backend, and the latter as the Client, or frontend. Clients and servers exchange request and response packages over a computer network.

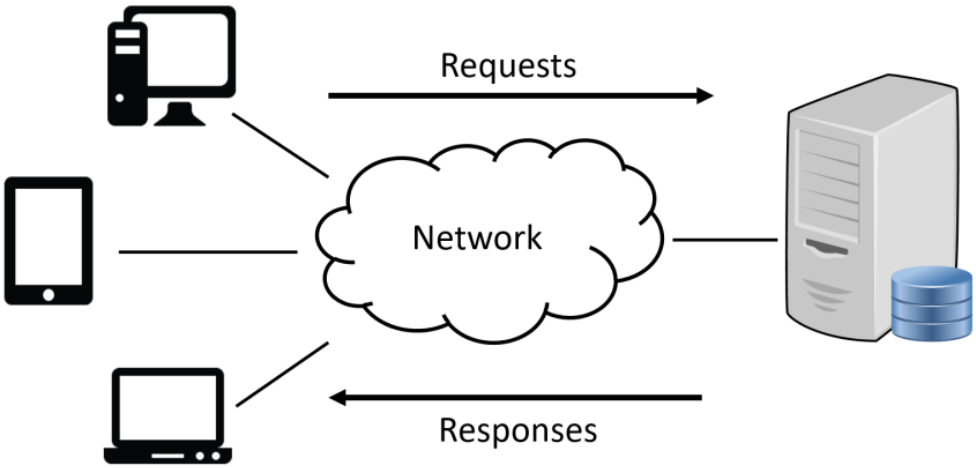


FIGURE 5.1 Client-server model

In our context, our server is the the CentOS virtual machine. Our platform will communicate directly to the services we need from the machine through API requests.

5.3 Quality requirements

The objective of this project is to develop a web-based network management system for Xtremesh device. In terms of quality and functionality, the application must meet the predefined requirements. If additional functions are required, they can be added at any time. The quality requirements of the system are:

- The system's user interface must be uncluttered and simple, with readable font and font-size.
- It must be responsive on both desktop and mobile browsers.
- Loading indicators for asynchronous items should be added.
- The system must not cause the browser to crash or freeze.

5.4 Platform architecture design

This section describes the Designing step in detail. the architecture is built around the requirements that have been previously defined. Fundamentally, it plays the role of a client-side application and employs React-redux as a development framework.

5.4.1 System design

This web-app's system design is based on the traditional client-server model.xMeshControl uses APIs methods and headers to send requests to the server, to which the specified service gets the call and responds with the appropriate response.

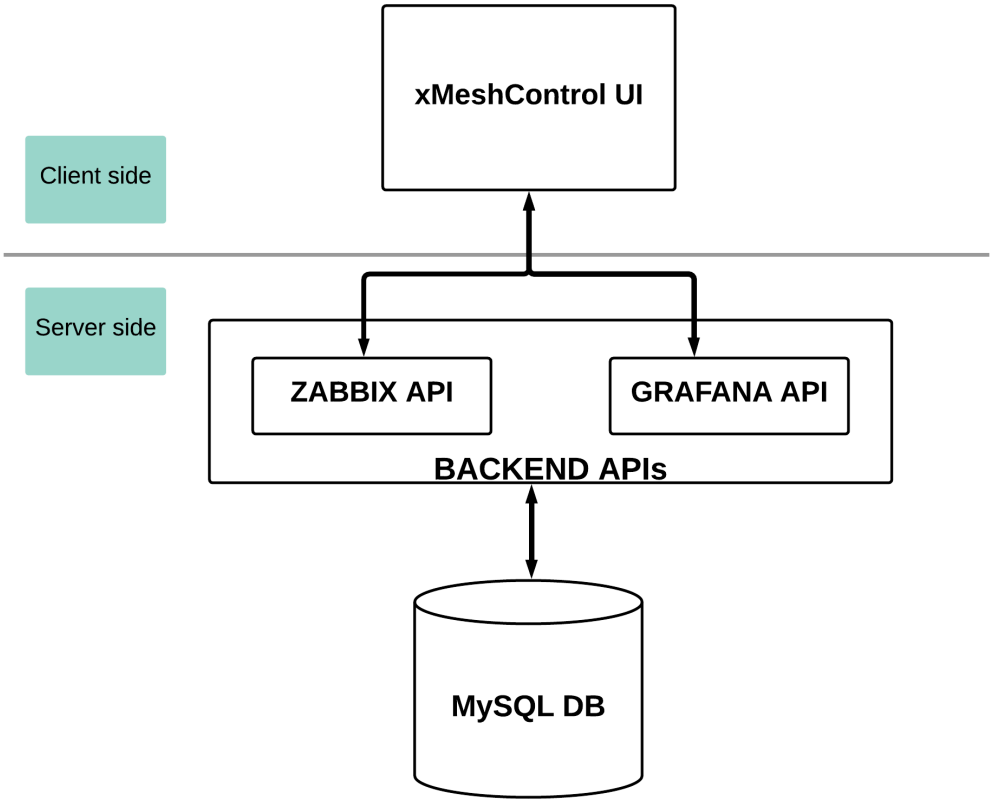


FIGURE 5.2 System design

The back-end APIs handles the requests and send back responses. Front-end do include calculations and structured data storage methods. However, it is primarily concerned with providing the best UI with the best performance and user experience.

5.4.2 Modular design

A modular design approach divides a system into various independent modules. A modular system has components with low coupling and high cohesion, which increases flexibility, modifiability, scalability, and reusability. It is possible to identify critical components in functionality and performance using various flow path analyses. It is possible to improve efficiency and performance by optimizing those components or replacing them with higher performing ones.

The first step in partitioning the system into discrete modules is to describe the various functions the system possesses in order to separate main components based on functionality. To analyze different flow paths and identify these critical components, use case diagrams can be used. Use case analysis is a valuable and important requirement analysis technique that is widely used in modern software engineering.

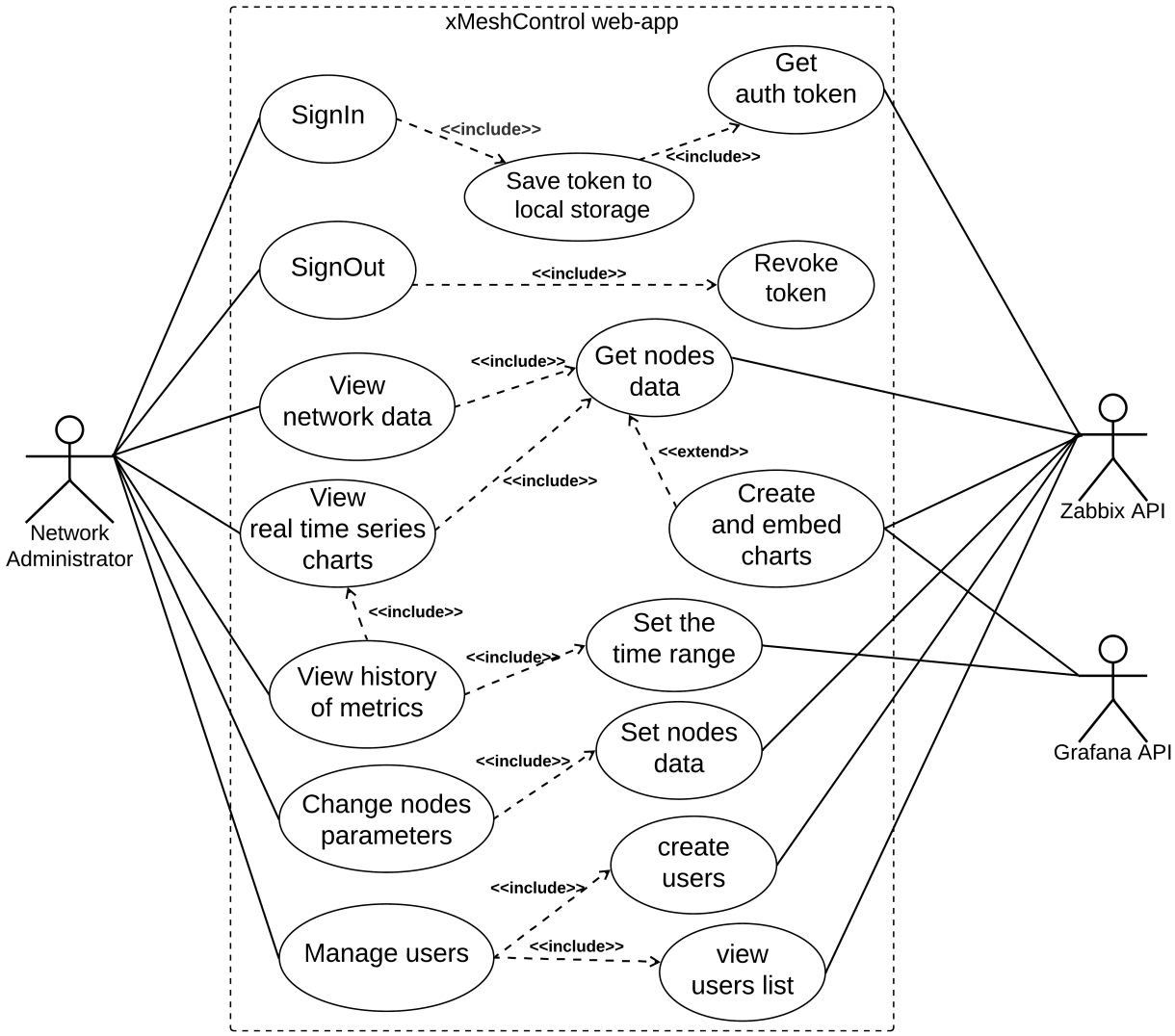


FIGURE 5.3 Use case diagram

The illustration above depicts the most fundamental interactions between the xMeshControl user and the back-end APIs. The xMeshControl components are sorted in this diagram based on the function they provide. They are divided into these major functionalities:

5.4.3 Authentication

xMeshControl uses Zabbix API authentication, that allows to log in to the API and generate an authentication token. A user attempts to login through the login screen, and a 'user.login' method is invoked to the Zabbix API. if the login is success, an auth token is generated as a response from the API, else Zabbix API returns an error message as a response. The auth token is then stored locally as it is mandatory for every request sent to the Zabbix API. A logout happens by simply removing the auth token.

In the root app file, I set it to check the value of the stored authToken before rendering the page. If the authToken is null or undefined, the app returns the login screen. Else if the token is generated, our application pages gets rendered.

Dynamic authentication process

This diagram explains how the authentication is handled in the web app using the Zabbix API authentication token, and therefore the state of requests handling depending on the token state.

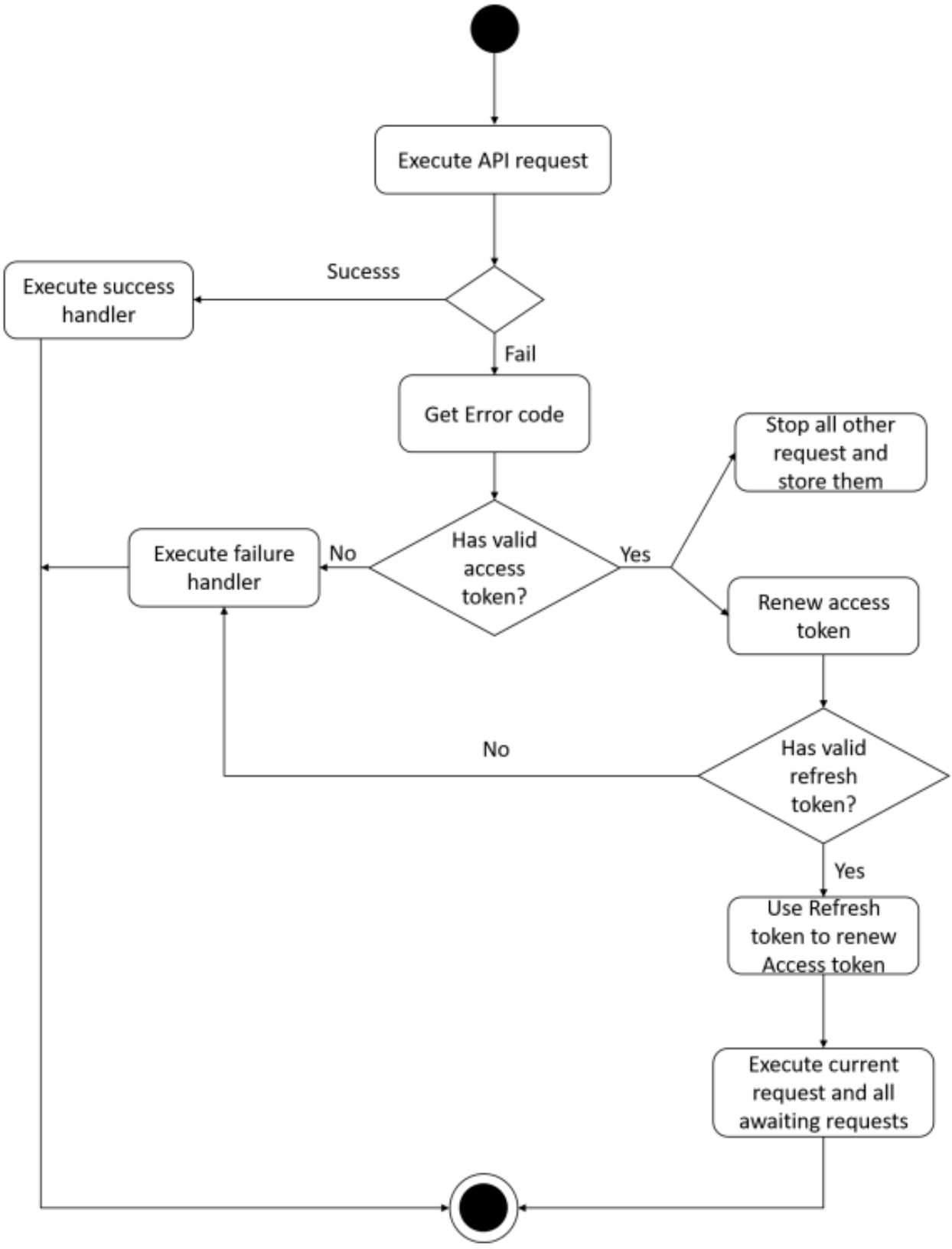


FIGURE 5.4 Authentication process

5.4.4 Grafana charts automation

As I have mentioned before, the application logic will handle the process of the automation of creating and embedding Grafana's time-series charts, by manipulating the JSON model of the dashboard template we created earlier, along with the help of the HTTP methods provided by Grafana's API.

So basically the idea i came up with can be described as follows:

1. A device unique ID in this process will be its IP address without the dots. For example, the host uid for the device with IP '192.168.1.137' is '1921681137'.
2. Code a function that returns the HTTP request body for the dashboard creation API method, that takes as an argument a hostUid (represents the device unique ID). We have now an API request body that can change depending of the parameter given. This method will be used to create a new dashboard for every new discovered Xtremesh host, if Zabbix adds the host to be monitored, this code helps in creating a new dashboard for it in Grafana.

```
export const CREATE_DASH_API_BODY = (hostUid) => (
    {
        "dashboard": {
          "id": hostUid,
          "uid": hostUid,
          "title": `xtremesh${hostUid}`,
          "timezone": "browser",
          "schemaVersion": 16,
          "version": 0,
          "refresh": "25s"
        },
        "folderId": 0,
     //   "folderUid": 'xtreMeshFolder',
        "message": "Updated !",
        "overwrite": false
    }
)
```

FIGURE 5.5 Request body function with host uid argument

The returned object is then used in this function that handles the dashboard creation API.

```
export async function createGrafanaDashboard(hostUid) {
    try {
        const res = await fetch(`${ZABBIX_HOST}:8080/grafana/api/dashboards/db`, {
            method: 'post',
            //mode: 'no-cors',
            headers: {
                'Accept': 'application/json',
                'Authorization': `Bearer ${GRAFANA_API_KEY}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(CREATE_DASH_API_BODY(hostUid))
        });
        const data = await res.json()
        return data
    } catch (error) {
        console.log(error)
    }
}
```

FIGURE 5.6 Dahboard creation API handler function

3. Code a function that returns the HTTP request body for populating the dashboard with our JSON model, this JSON model will be parametered so it can be only applied to a unique host with the appropriate unique dashboard created for that host. The parameters of this function are the response details from the creating-dashboard HTTP request.

```
export const UPDATE_DASH_API_BODY = (id, uid, title, host, version) => ({
  "dashboard": {
    "id": id,
    "panels": [
      {
        "datasource": null,
        "fieldConfig": {
          "defaults": {
            "color": {
              "mode": "palette-classic"
            },
            "custom": {
              "axisLabel": "",
              "axisPlacement": "auto",
              "barAlignment": 0,
              "drawStyle": "line",
              "fillOpacity": 0,
              "gradientMode": "none",
              "hideFrom": {
                "legend": false,
                "tooltip": false,
                "viz": false
              },
              "lineInterpolation": "smooth",
              "lineWidth": 2,
              "pointSize": 5,
              "scaleDistribution": {
                "type": "linear"
```

```
    },
    "timepicker": {
      "refresh_intervals": [
        "1s",
        "5s",
        "10s",
        "30s",
        "1m",
        "5m",
        "15m",
        "30m",
        "1h",
        "2h",
        "1d"
      ]
    },
    "timezone": "browser",
    "title": title,
    "uid": uid,
    "version": version
  }
})
```

FIGURE 5.7 Function that returns a parmetered JSON dashboard model

The returned object is then used here to handle the update dashboard API call.

```
export async function fillGrafanaDashboard(id, uid, title, host, version) {
    try {
        const res = await fetch(`${ZABBIX_HOST}:8080/grafana/api/dashboards/db`, {
            method: 'post',
            //mode: 'no-cors',
            headers: {
                'Accept': 'application/json',
                'Authorization': `Bearer ${GRAFANA_API_KEY}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(UPDATE_DASH_API_BODY(id, uid, title, host, version))
        });
        const data = await res.json()
        return data
    } catch (error) {
        console.log(error)
    }
}
```

FIGURE 5.8 Dashboard updating API handler function

4. In the app.js, the root file, I have coded a function 'makeDashboard()' to handle the process of the automation of the creation of dashboards upon a node discovery by Zabbix. The function checks an array of Xtremesh IPs discovered, loop through the array and proceed to create a dashboard for each of them with the create dashboard API handler function. if a dashboard is already present with the unique id specified, the API response returns an error message and it does not duplicate a dashboard with the same host uid. After creating a dashboard successfully, a success propriety with the value of 'true' comes in the response message of the creation API method request.

```
CREATING MESSAGE:                                                    App.js:80
▼{id: 81, slug: "xtremesh1921681138", status: "success", uid: "1921681138", url: "/grafana/d/1921681138/xtremesh1921681138", …}
   id: 81
   slug: "xtremesh1921681138"
   status: "success"
   uid: "1921681138"
   url: "/grafana/d/1921681138/xtremesh1921681138"
   version: 1
  ▶[[Prototype]]: Object
```

FIGURE 5.9 Dashboard creation success response

the function checks if that success value is true then calls the update dashboard API and fill the dashboard with our wanted charts specified in our JSON model. The parameters of the fill dashboard API handler function are the proprieties from the creation dashboard response.

5.4.5 Project Assessment

This section provides an evaluation of the project's outcome based on the defined requirements outlined in the project objectives.

1. The figure 5.10 illustrates the dashboard page of the platform. It focuses on the clear reading of network major statistics, each device link status and information in a paginated table. A graph displaying nodes layout and interconnection is also presented in the dashboard. Time series graphs highlighting different data plane statistics are also embedded.

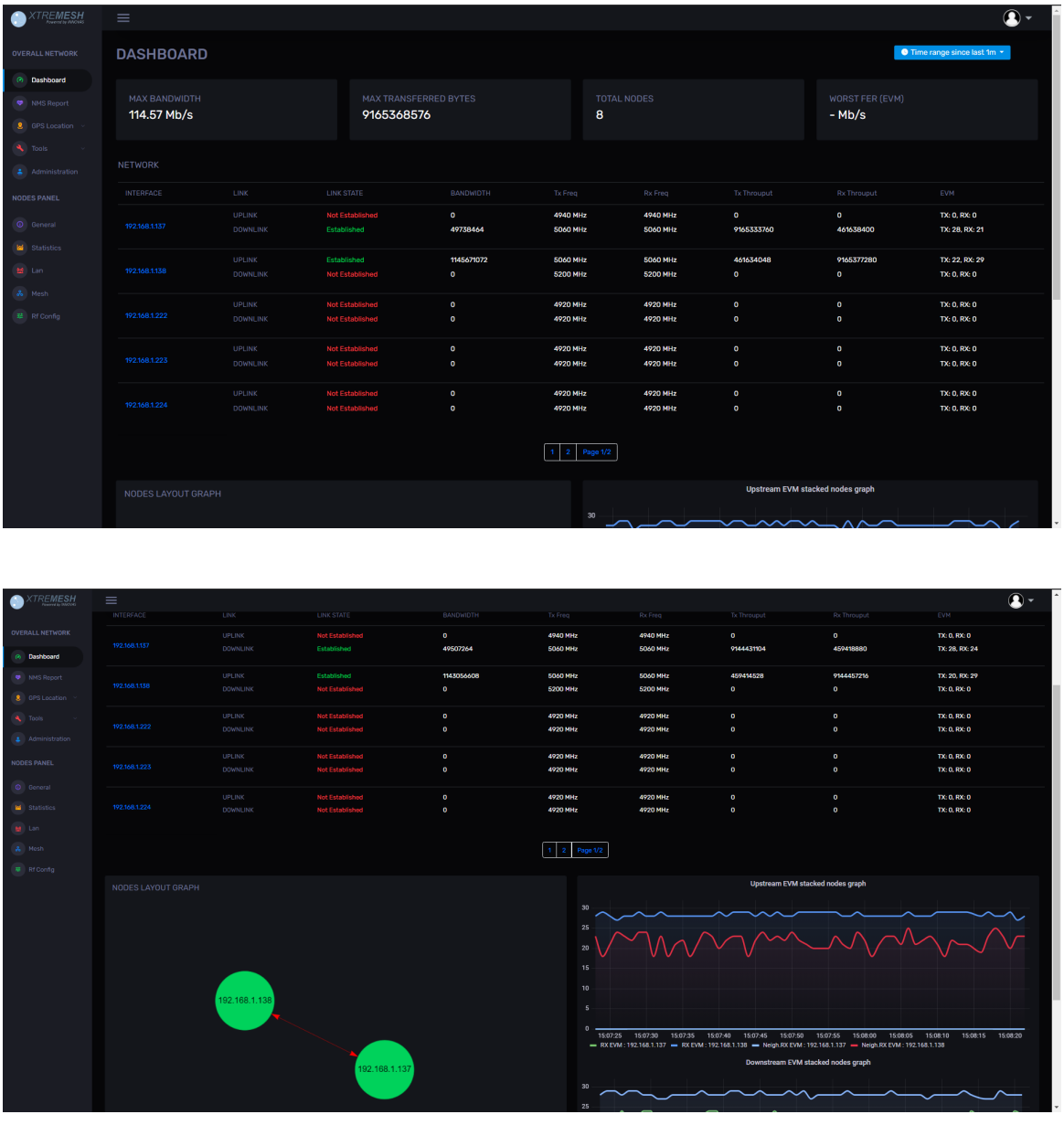


FIGURE 5.10 Dashboard page

2. This figure 5.11 presents the overall report of the Zabbix server in order to keep an eye on the server performance curves. each panel presents metrics such as traffic flows, cpu usage, memory usage, etc...

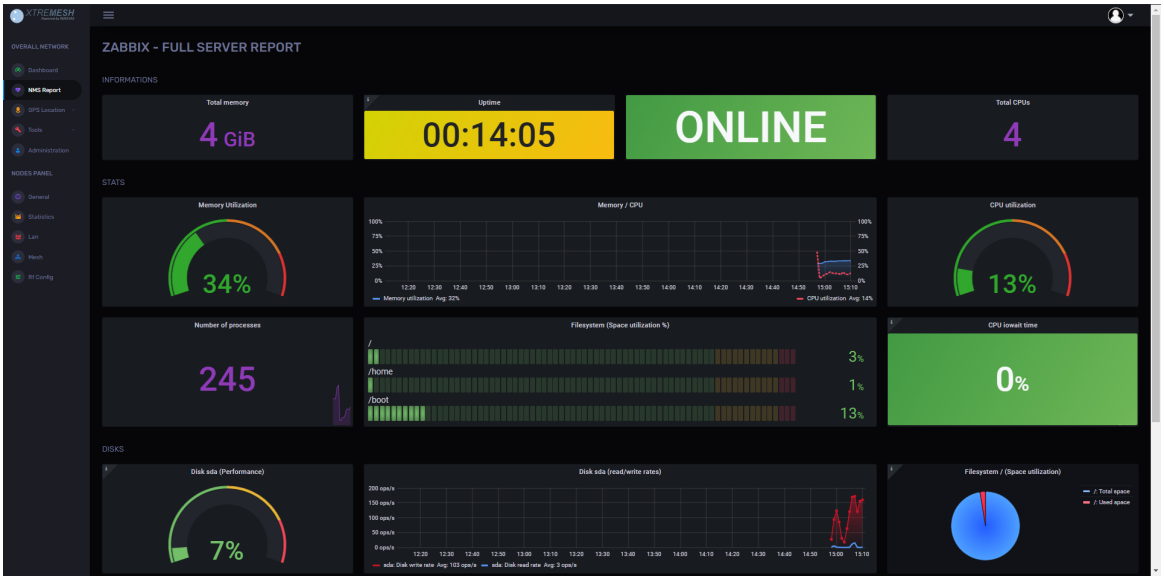


FIGURE 5.11 Zabbix report page

3. Nodes graphic visualization page is shown in the figure 5.12. In this page a user can check the status of links and interconnections between Xtremesh nodes, and also highlight a node to check its general information.

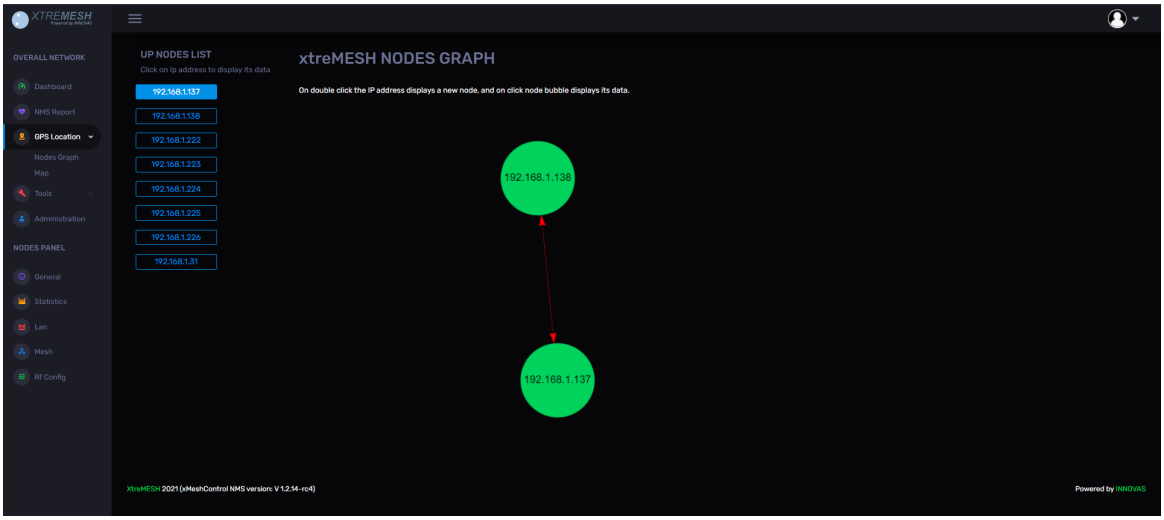


FIGURE 5.12 Zabbix report page

4. Users page is the section showed in the figure 5.13 is where the network administrator creates new users, and view their activities log in a table.

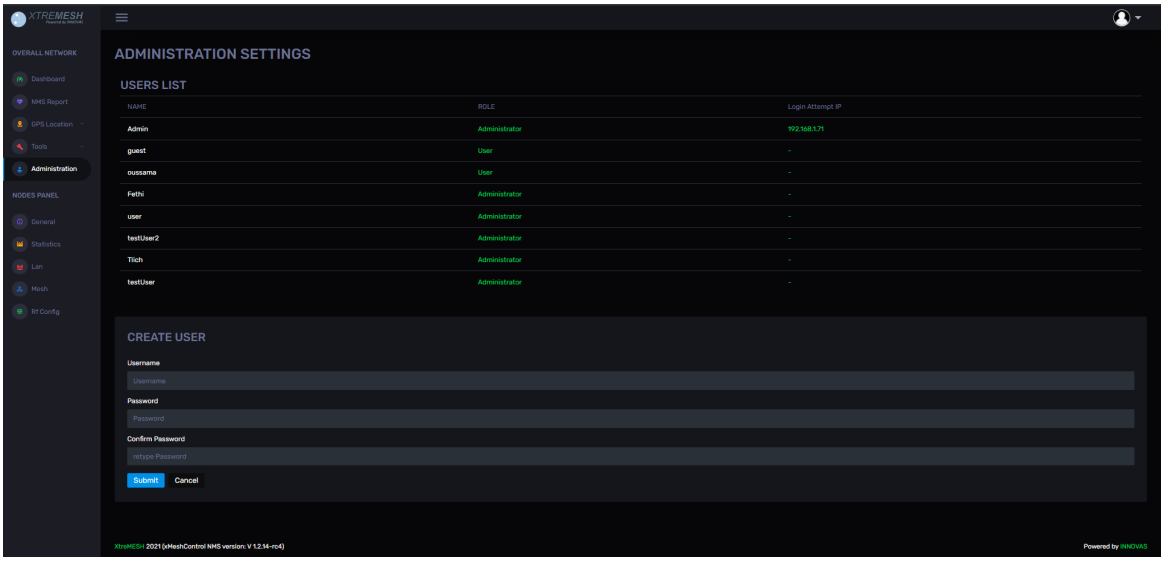


FIGURE 5.13 Users page

5. In this page, as shown by figure 5.14, a list of Xtremesh nodes are displayed at the left side, the user clicks on the wanted node's IP address to check its general informations.

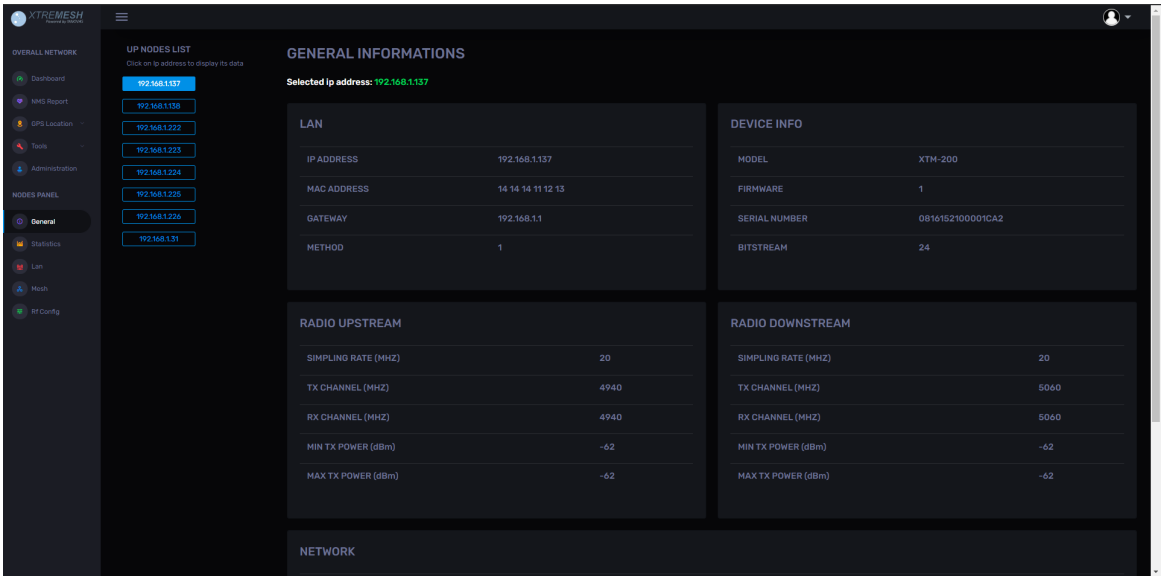


FIGURE 5.14 General Information page

6. This page in figure 5.15 is where real time data visualization of different metrics is implemented. A user selects the wanted device and more than 30 items can be monitored in a easy-to-use interactive UI. Checking previous and historical data up to 7 days is also possible by selecting the desired time range of data displays.

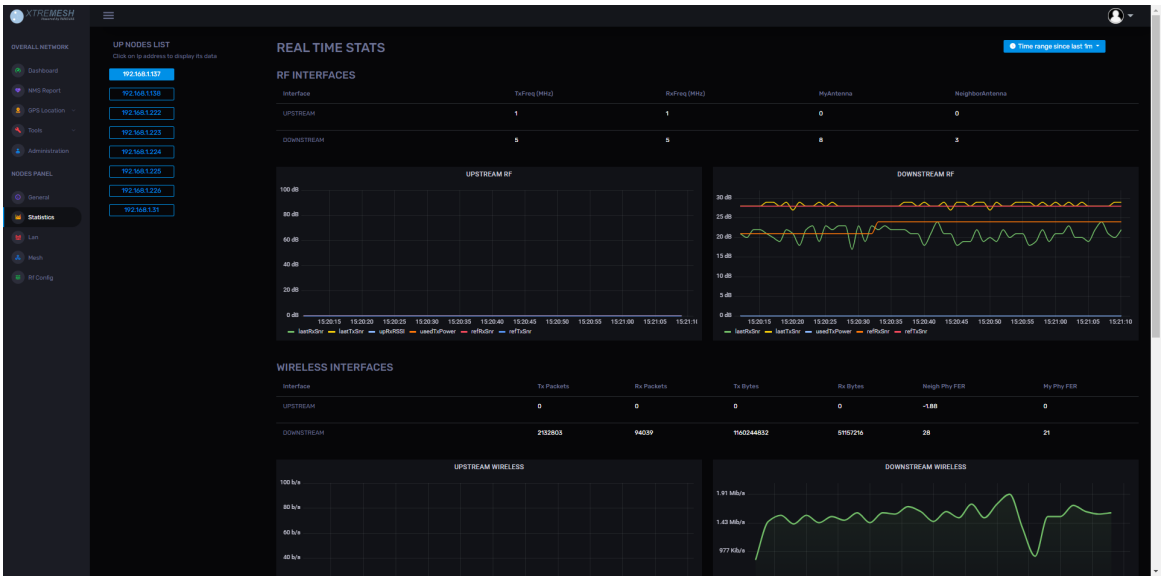


FIGURE 5.15 Real time statistics page

7. The figure 5.16 below shows the section where the user can set RF configuration for a specific device chosen from the list at the left.

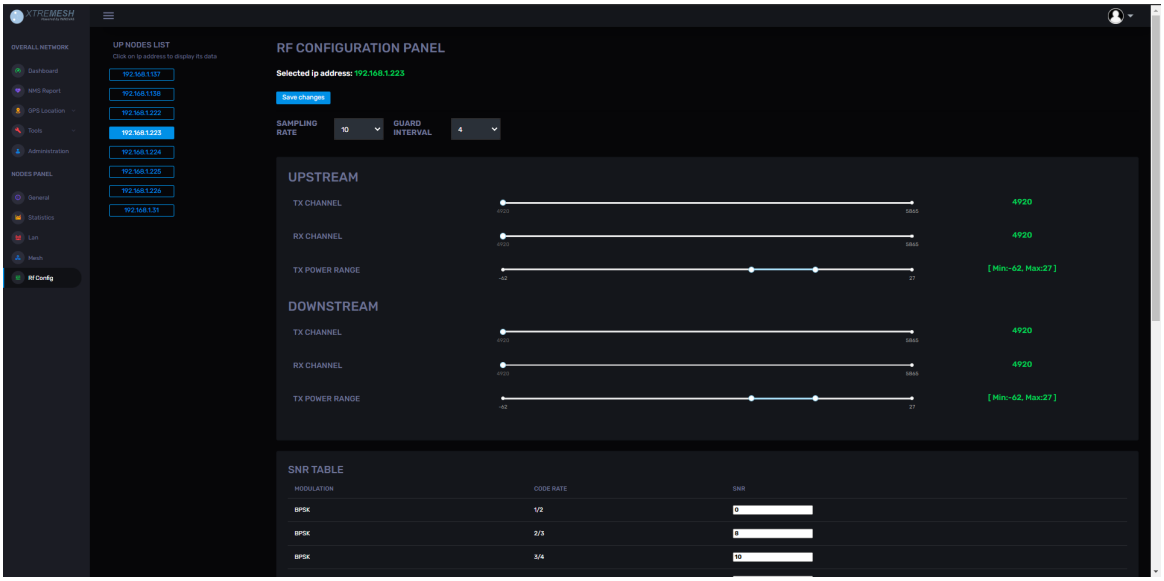


FIGURE 5.16 RF configurations page

5.5 Conclusion

In this chapter, I described the implementation process of the Xtremesh web platform, as well as illustrating the development process and the output with some screenshots.

GENERAL CONCLUSION

The adaptation of a network monitoring tool must be done with caution. The network administrator should be confided in about the network structure and should have a thorough understanding of the network devices and components that will be monitored. It should be noted that some features may result in a decrease in network performance. This project presents how CodinTek provisions its proper monitoring solution that adapts to the Xtremesh infrastructure and scale to its future visions. I am pleased to announce that, with a great deal of effort, patience, and attention, I was able to complete all stages of my project. This project has a lot of room for improvement, particularly in terms of optimization and adding variety of features such as generating logs, notification system and security. We intend to improve the system's security in the future by encrypting SNMP traffic via SSH tunneling, because SNMP requests can be intercepted and altered by unauthorized users. Finally, this work has provided a significant opportunity to gain a deeper understanding of network monitoring and web development, as well as understanding at high level of abstraction network behaviour and performance curves.

REFERENCES

[1] Codintek, *Technology & Products*
https://www.codintek.com/
visited on : 5/2021.

[2] Manage engine, *Basics of Network Monitoring*
https://www.manageengine.com/network-monitoring/
visited on : 5/2021.

[3] Chuck Moozakis, *Network Management System*
https://www.techtarget.com/searchnetworking/definition/network-management-system
visited on : 6/2021.

[4] Eginnovations, *It's Not about Agent vs. Agentless Monitoring Anymore*
https://www.eginnovations.com/blog/agentless-vs-agent-based-monitoring/
visited on : 8/2021.

[5] Instana, *Automatic Discovery Monitoring*
https://www.instana.com/docs/auto_discovery/
visited on : 8/2021.

[6] Anatolii Shokhin, *Network Monitoring with Zabbix*
published on : 6/2015
Address: University of Applied Science.

[7] Compare Camp, *Zabbix Review*
https://comparecamp.com/zabbix-review/
visited on : 6/2021.

[8] Capterra, *OpenNMS by The OpenNMS Group*
https://www.capterra.com/p/176635/OpenNMS/
visited on : 6/2021.

[9] Icinga, *Monitor Your Entire Infrastructure*
https://icinga.com/
visited on : 6/2021.

[10] Zabbix, *Zabbix documentation*
https://www.zabbix.com/manuals
visited on : 6/2021.

[11] Grafana, *About Grafana*
https://www.grafana.com
visited on : 6/2021.

[12] Geeks For Geeks, *MySQL, common MySQL queries*
https://www.geeksforgeeks.org/mysql-common-mysql-queries/
visited on : 8/2021.

[13] Geeks For Geeks, *What is Nginx and how to install it*
https://www.geeksforgeeks.org/what-is-nginx-web-server
visited on : 8/2021.

[14] Auvik, *Network basics: What is SNMP*
https://www.auvik.com/franklyit/blog/
network-basics-what-is-snmp/
visited on : 8/2021.

[15] Thwak, *What are MIBs and OIDs? How to manage them?*
https://thwack.solarwinds.com/resources/b/
geek-speak/posts/what\protect\@normalcr\
relax-are-mibs-and-oids-how-to-manage-them
visited on : 8/2021.

[16] React, *React, A JavaScript library for building user interfaces*
https://reactjs.org/
visited on : 8/2021.